COBOL

A Translator from COBOL to ANSI C (C89)

Introduction and User's Guide

Release 1

for VM/370 CMS and Linux

Publication COB-0001-0

First Edition (October 2026)

This edition applies to cobol Release 1 (October 2026), a translator from COBOL to ANSI C (C89) written in C89, and to all subsequent versions until otherwise indicated in new editions.

cobol is a hobbyist project. It implements a large working subset of COBOL, with some features of later dialects and object-oriented COBOL, and is not an implementation of any particular commercial product. The language is described against COBOL-85 (ANSI X3.23-1985), with selected features of COBOL 2002, 2014 and 2023.

IBM, COBOL (CODASYL and ISO) and GnuCOBOL are names of products or specifications of their respective owners and are used here only to identify them. Other names are trademarks of their respective owners.

The project repository holds the translator source, the run-time header, the test programs, the examples and this manual, and every example in the manual can be run from it. Comments on this publication may be addressed to the project.

Contents

Preface **1**

Chapter 1. Introduction **3**
- What cobol is 3
- How it works 3
- Platforms 4
- The 71-column rule 4
- What it can do 4
- What it cannot do 4

Chapter 2. Installing and Running **5**
- Building the translator 5
- Translating and running a program 5
- The command line 5
- Choice of the C compiler 6
- Several source files 7
- Environment variables 7
- Input and output files 8
- Return codes 8

Chapter 3. Source Format **9**
- Fixed format 9
- Free format 11
- Words, literals and separators 11
- The COPY statement 12
- Compiler directives 13

Chapter 4. Program Structure **15**
- The divisions 15
- Paragraphs, sections and sentences 15
- Programs in one file 17
- Special registers 18

Chapter 5. The Data Division **19**
- Data descriptions 19
- The PICTURE clause 19
- The USAGE clause 21
- VALUE and REDEFINES 22
- Tables: OCCURS 23
- Condition names 23
- Constants 24
- Other clauses 25

Chapter 6. Procedure Division Statements **26**
- DISPLAY and ACCEPT 26
- MOVE 27
- Arithmetic 28
- Choosing: IF and EVALUATE 30
- PERFORM 31
- GO TO and EXIT 32
- STRING, UNSTRING and INSPECT 33
- SET, INITIALIZE, ALLOCATE and FREE 34

Chapter 7. Conditions and Expressions **36**
- Arithmetic expressions 36
- Conditions 36

Chapter 8. Tables **39**

Subscripts and indexes....39
Tables of variable length....40
SEARCH....40
Sorting a table....41

Chapter 9. Intrinsic Functions....43
String functions....44
Numeric functions....45
Date functions....45
Functions with limits....46

Chapter 10. Files....47
The SELECT entry....47
How the files are stored....47
The statements....48
The file status....52
Declaratives....54

Chapter 11. Sorting and Merging....56

Chapter 12. Subprograms, User Functions and C....59
CALL....59
User-defined functions....60
Separate modules....61
Calling C functions....62

Chapter 13. Object-Oriented COBOL....64
Classes and methods....64
Factory methods and overriding....66

Chapter 14. Diagnostics and Run-Time Errors....69
Messages of the translator....69
Run-time errors....70
The exit status....71

Chapter 15. Implementation Notes and Limits....72
The decimal engine....72
Data representation....72
Limits....72
What is not implemented....73
Known defects....73
Files of the translator....73

Appendix A. Words to Avoid....74

Appendix B. GnuCOBOL Compatibility....76

Appendix C. Running on VM/370 CMS....77

Preface

This manual introduces cobol, a translator from COBOL to ANSI C (C89). It tells you how to build the translator, how to turn a COBOL program into a running program, and what the translator understands: the divisions of a program, the data description, the statements, the file handling, sorting, subprograms, the call interface to C, and object-oriented COBOL.

COBOL, the Common Business-Oriented Language, was designed in 1959 for business data processing. Its data is described by pictures, so that a field is a number of decimal digits or characters with a fixed place for the decimal point, and its arithmetic is decimal arithmetic: 0.1 plus 0.2 is exactly 0.3. cobol keeps to this. It carries out all arithmetic with its own exact decimal engine and never converts a number to binary floating point unless the program asks for a COMP-1 or COMP-2 field.

The translator is written in C89 and its output is C89. It was designed to build with GCC on VM/370 CMS (the VM/370 Platinum, or VM/370plus, system) and with any C compiler on Linux. All source lines of the translator and of the run-time header are 71 columns or shorter, so that they fit an 80-column card image with room for line numbers. The language is the COBOL-85 of ANSI X3.23-1985 with a selection of the features of COBOL 2002, 2014 and 2023, among them free-format source, user-defined functions, intrinsic functions, the BINARY-xxx data types and object-oriented classes.

The manual is written for the reader who knows some programming language, but not necessarily COBOL. Chapters 3 to 8 describe the language, Chapter 9 lists the intrinsic functions, and Chapters 10 to 13 describe files, sorting, subprograms and objects. The examples can be typed in as they stand, and they are also in the directory `examples` of the repository.

How this manual is organized

- **Chapter 1, Introduction,** says what cobol is and how it works.
- **Chapter 2, Installing and Running,** takes you from the source to a running program, and lists the options and environment variables.
- **Chapter 3, Source Format,** describes fixed and free format, continuation lines, COPY, REPLACE and the compiler directives.
- **Chapter 4, Program Structure,** describes the divisions, paragraphs and sections, and nested programs.
- **Chapter 5, The Data Division,** describes pictures, usages, tables, condition names and the other clauses of a data description.
- **Chapter 6, Procedure Division Statements,** describes the statements, one by one.
- **Chapter 7, Conditions and Expressions,** describes arithmetic expressions and the conditions of IF, EVALUATE and PERFORM.
- **Chapter 8, Tables,** describes subscripts, indexes, OCCURS DEPENDING ON, SEARCH and sorting a table.
- **Chapter 9, Intrinsic Functions,** lists every function that is implemented.
- **Chapter 10, Files,** describes sequential, line sequential, relative and indexed files, the file status codes and the declaratives.
- **Chapter 11, Sorting and Merging,** describes SORT and MERGE.
- **Chapter 12, Subprograms, User Functions and C,** describes CALL, LINKAGE, dynamic modules and calling C functions.
- **Chapter 13, Object-Oriented COBOL,** describes classes, methods and INVOKE.
- **Chapter 14, Diagnostics and Run-Time Errors,** lists the messages.
- **Chapter 15, Implementation Notes and Limits,** describes the decimal engine and what is not implemented.
- **Appendixes** list the words to avoid, compare cobol with GnuCOBOL and describe the use on VM/370 CMS.

Conventions

Program text, file names and commands are shown in monospaced type. COBOL does not distinguish upper and lower case in names or keywords: `Total`, `TOTAL` and `total` are the same name. This manual writes the keywords and the names of fixed-format programs in upper case, as COBOL programs always did. Examples of programs are shown in framed figures. Fixed-format examples are shown exactly as they are in the file, so that you can see which text is in which column. A line that begins with `$` is a command typed at the shell prompt, and the lines after it are what the command or the program wrote. Every example was translated and run by the translator described here, and the output shown is the real output.

A **sentence** ends with a period. In the syntax descriptions, items in square brackets are optional and a vertical bar separates alternatives. The word *translator* means the program `cobol`; the word *runtime* means the header file `cobrt.h` that the generated C program includes. The word *field* means any data item that holds a value.

Chapter 1. Introduction

What cobol is

cobol reads one or more COBOL source programs and writes a C program. The C program is compiled with an ordinary C compiler and linked with the C math library. Nothing else is needed: the COBOL run-time system is the single header file `cobrt.h`, which the generated program includes and which consists of `static` functions only. The translator is one C file, or fifteen files that are joined into one by the script `tools/amalg.sh`, and it can start the C compiler itself.

Step	Program	Input	Output
1	`cobol`	The COBOL source, `prog.cob`	The C program, `prog.c`
2	C compiler	`prog.c` and `cobrt.h`	The program `prog`
3	`prog`	Files and standard input	The output of the program

Figure 1. From COBOL source to output

With the option `-c` the translator performs steps 1 and 2 in one command. The smallest complete program, `examples/hello.cob`, shows the whole path.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HELLO.
      * The smallest useful program: a greeting and a little arithmetic.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WHO       PIC X(12)  VALUE "WORLD".
       01 N1        PIC 9(3)   VALUE 17.
       01 N2        PIC 9(3)   VALUE 25.
       01 TOTAL     PIC 9(4).
       PROCEDURE DIVISION.
       MAIN-PARA.
           DISPLAY "Hello, " FUNCTION TRIM(WHO) "!".
           ADD N1 N2 GIVING TOTAL.
           DISPLAY N1 " + " N2 " = " TOTAL.
           IF TOTAL > 40
               DISPLAY "that is more than forty"
           ELSE
               DISPLAY "that is forty or less"
           END-IF.
           STOP RUN.

$ ./hello
Hello, WORLD!
017 + 025 = 0042
that is more than forty
```

Figure 2. The first program

How it works

Every COBOL program becomes one C function. Every data item of the program becomes a small descriptor, a `CobFld`, which holds the address and the size of the item in the storage of its record and a pointer to an attribute record that says what kind of field it is: how many digits, where the decimal point is, whether it is signed, which picture. The statements of the procedure division become calls of run-time functions that take such descriptors, and the paragraphs become labels in the function; PERFORM and GO TO become jumps. This design keeps every COBOL storage rule: a group is the concatenation of its members, REDEFINES overlays storage, and MOVE and comparison work on the bytes in the way the standard describes.

Arithmetic is exact. A number is held in the run-time system as a string of up to 80 decimal digits with a sign and a scale, and addition, subtraction, multiplication, division, comparison and exponentiation are done on these strings, so that no 64-bit integer type is needed and results do not depend on the machine. Chapter 15 describes the engine and its limits.

The run-time system sees an EBCDIC machine as well as an ASCII machine: the sign zones of zoned decimal numbers and the packed decimal formats are chosen at compile time from the character set of the C compiler.

Platforms

cobol builds with GCC on Linux and with any C89 compiler. The translator and the run-time header use no `long long`, no `stdint.h` and no other C99 feature, and the output uses none either, so it is meant to build with the GCC of VM/370plus as well. The command `cobol` can start gcc, cc, clang, TinyCC and, on Windows, Microsoft C. The translator and the programs it makes have been built and tested with GCC on Linux. On VM/370 CMS the generated program is compiled by GCC like any other C program; Appendix C describes the steps. The dynamic loading of modules (Chapter 12) needs `dlopen`, which Linux and other Unix systems have, and is left out where it is not available.

The 71-column rule

All the source text of the translator, the run-time header and the tests is at most 71 columns wide, so that it can be sent to a CMS minidisk with fixed 80-column records and a line number in columns 73 to 80 without loss. This rule applies to the translator, not to your programs. Your fixed-format COBOL programs follow the usual rule of the language: columns 1 to 6 are the sequence area, column 7 is the indicator area, columns 8 to 72 hold the program text, and the text after column 72 is ignored. A longer line is silently cut at column 72, which is the most common cause of surprising syntax errors in fixed-format programs. Chapter 3 shows an example.

What it can do

- The four divisions, sections and paragraphs, nested programs, and the program attributes COMMON, INITIAL and RECURSIVE.
- Data of every usage of COBOL-85: DISPLAY, COMP and BINARY, COMP-3 and PACKED-DECIMAL, COMP-1 and COMP-2, and edited pictures, and of COBOL 2002: COMP-5, BINARY-CHAR to BINARY-DOUBLE, FLOAT-SHORT to FLOAT-EXTENDED, INDEX, POINTER, NATIONAL, boolean pictures and OBJECT REFERENCE.
- The clauses REDEFINES, OCCURS (also DEPENDING ON), JUSTIFIED, BLANK WHEN ZERO, SIGN, SYNCHRONIZED, CONSTANT, EXTERNAL, GLOBAL and BASED, and condition names.
- All the arithmetic statements with ROUNDED, SIZE ERROR and CORRESPONDING, the COMPUTE statement with exponentiation, and the rounding modes of COBOL 2014.
- IF, EVALUATE, PERFORM in every form (also inline, VARYING, TEST AFTER and EXIT PERFORM), GO TO DEPENDING, STRING, UNSTRING, INSPECT, SET, SEARCH, INITIALIZE, ACCEPT and DISPLAY, CALL, CANCEL, ALLOCATE and FREE.
- Sequential, line sequential, relative and indexed files, with alternate keys, FILE STATUS, USE declaratives, SORT and MERGE with input and output procedures, and the sorting of tables.
- Nearly one hundred intrinsic functions, and user-defined functions with FUNCTION-ID.
- Calls between programs of one file, of several files, and of dynamic modules, and calls of C functions.
- Classes with FACTORY and OBJECT parts, methods, INHERITS, INVOKE with SELF and SUPER, and object references.
- COPY with REPLACING, REPLACE, conditional compilation with `>>IF`, and free-format source.

What it cannot do

The report writer, the screen section, the communication section, ALTER, the JSON and XML statements, ENTRY, and the SORT and MERGE of files on several devices are not implemented. SEARCH ALL is a linear scan, not a binary search. Chapter 15 has the complete list of restrictions.

Chapter 2. Installing and Running

Building the translator

The repository contains the translator source in `src`, the tools in `tools`, and a Makefile. Any C compiler will do; the Makefile asks for `-std=c89 -pedantic`, which is what the test suite uses. The Makefile joins the source files into `cobol_all.c`, compiles that into the program `cobol`, and builds the run-time header `src/cobrt.h` from its parts.

```
$ git clone https://github.com/mikebeer/cobol
$ cd cobol
$ make              # builds ./cobol and src/cobrt.h
$ make test         # translates, builds and runs every test program
$ make lines        # checks that no source line is longer than 71 columns
```

Figure 3. Building and testing the translator

Translating and running a program

The command `cobol` takes the names of one or more source files and options. A file name without a file type gets the first of `cob`, `cbl`, `cobol`, `COB` or `CBL` that exists, so `cobol hello` reads `hello.cob`. Without `-c` the translator only writes the C program, named like the source with the type changed to `c`. It is written next to the source file, and stays there after a build, so run the translator in a work directory if you do not want that.

```
$ cobol hello.cob
$ ls
hello.c
hello.cob
$ cc -std=c89 -w -Icobol/src -o hello hello.c -lm -ldl -rdynamic
$ ./hello
Hello, WORLD!
017 + 025 = 0042
that is more than forty
$ cobol -c hello.cob -o hello2
$ ./hello2
Hello, WORLD!
017 + 025 = 0042
that is more than forty
```

Figure 4. Translating a program in two steps and in one step

A C program that was made by `cobol` includes `cobrt.h`, so the C compiler must find that header: either name its directory with `-I`, or copy the header next to the program. When the translator starts the C compiler itself, with `-c`, it adds the directories next to the `cobol` program, its `src` directory, `../share/cobol` and `/usr/share/cobol` to the include path. The libraries `-lm`, `-ldl` and, so that a program can call functions of the C files linked with it, `-rdynamic` are added on Unix systems.

The command line

Option	Meaning
`-o file`	Name of the output. Without `-c` it is the C file; the default is the first source name with the type changed to `c`, and `-o -` writes the C program to standard output. With `-c` it is the program that is built; the default is the first source name without its type.
`-c`, `-x`	Translate, then build the program with a C compiler. `-x` is the same option under the name that other COBOL compilers use.
`-C`	Translate and compile the C program to an object file, but do not link. The object file is named like the C file with the type `o`.

Option	Meaning
-M	Make a module: no main program is generated. The programs of the source can be called with CALL from another program. With -c the result is a shared library (see Chapter 12).
-free	The source is in free format (Chapter 3). The directive >>SOURCE FORMAT IS FREE in the file has the same effect for the part of the file after it.
-fixed	The source is in fixed format. This is the default.
-I dir	Names a directory in which COPY looks for copybooks, after the directory of the source file and the current directory. -Idir is also accepted. The option can be given up to 32 times.
-Dname[=value]	Defines name for the conditional compilation directives >>IF and >>DEFINE. Without a value the value is 1.
-debug	Compiles the debugging lines, the lines that have D in column 7.
-check	Intended to check subscripts and reference modification at run time. In this release the option is accepted but has no effect; set the environment variable COB_CHECK when you run the program instead (see below).
-binsize 1-2-4-8	Selects the sizes of binary fields. With 1-2-4-8 a field of one or two digits takes one byte. The default 2-4-8 is the other choice: one to four digits take two bytes, five to nine digits four bytes, ten to eighteen digits eight bytes (Chapter 5).
-cc name	The C compiler for -c: a command such as gcc, cl for Microsoft C, or bundled for the TinyCC next to the cobol program.
--version	Prints the version of the translator and stops.

Figure 5. Options of the cobol command

Other names on the command line are source files if their type is cob, cbl, cobol or cpy, or if they have no type; everything else, such as a C file, an object file or a library, is handed to the C compiler together with the C program. An unknown option makes the translator write a summary of the options and stop with return code 4. The return code is 0 when the translation (and the build) succeeded, and 8 when the translator found an error in the source or the C compiler failed.

```
$ cobol
cobol 1.0 -- COBOL to C89 translator
usage: cobol [options] src.cob ...
  -o f      output file (- for stdout)
  -c        build the program (needs a C
            compiler); -o names the exe
  -C        compile to objects, no link
  -M        module: no main program
  -free     free-format source
  -fixed    fixed-format source (default)
  -I dir    copybook directory
  -Dn[=v]   define n for >>IF
  -debug    compile debugging lines (D)
  -check    check subscripts at run time
  -binsize 1-2-4-8|2-4-8   binary field sizes
  -cc c     C compiler, or 'bundled'
$ cobol --version
cobol 1.0
```

Figure 6. The option summary and the version

Choice of the C compiler

With -c the translator needs a C compiler. It looks for one in this order: the one named by -cc, the command in the environment variable COBOL_CC, then gcc, cc, clang, tcc and, on Windows, cl, tried in this order by running each with --version, and last the TinyCC in the directory tcc next to the cobol program. The word bundled after -cc forces the compiler in that directory. If none is found, the translator says so and stops with return code 8.

On Windows the translator and the programs it makes use the `.exe` file type, and the translator finds its own directory with the Windows interface. A TinyCC kept in a directory `tcc` next to `cobol.exe` is all that is needed to turn a COBOL program into an executable on a machine that has no other compiler: `cobol -c prog.cob` produces `prog.exe`. TinyCC is distributed under the GNU Lesser General Public License; keep its licence text in the `tcc` directory. Windows has no `dlopen`, so there a program can call only the programs that were translated together with it; dynamic modules and C functions cannot be called by name (Chapter 12).

Several source files

Several COBOL files can be given in one command. They are translated together into one C program in which all the programs can call each other by name; the first file names the C file and the program. Files of other types, such as C files, are compiled and linked with it.

```
      * main.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. SHOP.
      * The main program calls the subprogram TAXCALC.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 PRICE        PIC 9(5)V99 VALUE 250.00.
       01 TAX          PIC 9(5)V99.
       01 TAX-RATE     PIC 9V999   VALUE 0.077.
       01 SHOWN        PIC ZZ,ZZ9.99.
       PROCEDURE DIVISION.
           CALL "TAXCALC" USING PRICE TAX-RATE TAX.
           MOVE TAX TO SHOWN.
           DISPLAY "tax on 250.00 at 7.7 percent: " SHOWN.
           STOP RUN.

      * tax.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. TAXCALC.
      * Computes a tax amount, rounded to the cent.
       DATA DIVISION.
       LINKAGE SECTION.
       01 L-AMOUNT     PIC 9(5)V99.
       01 L-RATE       PIC 9V999.
       01 L-TAX        PIC 9(5)V99.
       PROCEDURE DIVISION USING L-AMOUNT L-RATE L-TAX.
           COMPUTE L-TAX ROUNDED = L-AMOUNT * L-RATE.
           GOBACK.

$ cobol -c main.cob tax.cob -o prog
$ ./prog
tax on 250.00 at 7.7 percent:     19.25
```

Figure 7. Two source files built into one program

Environment variables

Variable	Meaning
`COBOL_CC`	The C compiler that `cobol -c` should use, when `-cc` is not given.
`COB_LIBRARY_PATH`	At run time: the directories, separated by colons, in which CALL looks for dynamic modules, before it looks in the current directory (Chapter 12).
`COB_CHECK`	At run time: if the variable exists, subscripts and reference modifications are checked and an error ends the program with the message `subscript out of range`.

Variable	Meaning
DD_name	At run time: the file to which the file name name is mapped (see below). The names dd_name and name are also tried.

Figure 8. Environment variables

Input and output files

A file in COBOL is named in the SELECT clause. Its ASSIGN clause gives a literal, which is a name, or the name of a data item, whose value at the time of OPEN is the name. Before the file is opened, the run-time system looks at a name that has no slash or backslash in it: if there is an environment variable DD_name it uses its value as the real file name, then dd_name, then name. That is how a job step on the mainframe chose its files with a DD statement, and it lets you run the same program on different files without changing it. A name with a path is used as it is. On CMS the name is whatever the C library maps to a file; with PDPCLIB a FILEDEF for the name does the job.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT OUT-FILE ASSIGN TO "RESULT"
               ORGANIZATION IS LINE SEQUENTIAL.
       DATA DIVISION.
       FILE SECTION.
       FD OUT-FILE.
       01 OUT-REC PIC X(20).
       PROCEDURE DIVISION.
           OPEN OUTPUT OUT-FILE.
           MOVE "written by DEMO" TO OUT-REC.
           WRITE OUT-REC.
           CLOSE OUT-FILE.
           DISPLAY "done".
           STOP RUN.

$ DD_RESULT=answer.txt ./demo
done
$ cat answer.txt
written by DEMO
```

Figure 9. Choosing a file through the environment

Return codes

STOP RUN ends the program with the value of the special register RETURN-CODE as the exit status, which is 0 unless the program or a called program has set it. A call of a C function sets RETURN-CODE to the integer that the function returned. A run-time error writes `cobol: runtime error: message` to the standard error file and ends with the exit status 1.

Chapter 3. Source Format

A COBOL source file can be in fixed format, which is the format of the punched card, or in free format. Fixed format is the default. The option `-free`, or the directive `>>SOURCE FORMAT IS FREE` in the first line of the file, selects free format.

Fixed format

A line of a fixed-format program has four parts. The rule matters, because the translator really does ignore everything in columns 73 and beyond.

Columns	Name	Use
1 to 6	Sequence area	Ignored. Line numbers were punched here.
7	Indicator area	A blank for a normal line. `*` or `/` makes the line a comment (and `/` a page eject in a listing). `-` marks a continuation line. `D` or `d` marks a debugging line, which is compiled only with the option `-debug`.
8 to 11	Area A	Division, section and paragraph names, and the level numbers 01 and 77, begin here.
12 to 72	Area B	The rest of the program text. The translator does not enforce the difference between the areas A and B: a paragraph name in column 12 is accepted, but write programs in the traditional way.
73 and up	Identification area	Ignored.

Figure 10. The columns of a fixed-format line

```
         1         2         3         4         5         6         7
1234567890123456789012345678901234567890123456789012345678901234567890123456789012
000100 IDENTIFICATION DIVISION.
000200 PROGRAM-ID. RULER.
000300 PROCEDURE DIVISION.
000400 MAIN-PARA.
000500*    A comment line has an asterisk in column 7.
000600     DISPLAY "COLUMNS 12 TO 72 HOLD THE STATEMENT".
000700     STOP RUN.

$ ./demo
COLUMNS 12 TO 72 HOLD THE STATEMENT
```

Figure 11. The columns of a line

A tab character in the source is replaced by blanks up to the next column that is a multiple of eight, so that fixed-format sources written with a tab-friendly editor still work. A line that has fewer than seven characters is ignored.

Continuation lines

A literal that does not fit on one line is continued like this: the first line is filled up to column 72, without a closing quote; the next line has a hyphen in column 7, and its text, which is in area B, begins with a quote, followed by the rest of the literal. A first line that ends before column 72 is taken to continue to column 72 with blanks, so those blanks are part of the literal. The continuation of a word or of a numeric literal puts a blank in front of the continued text, so only literals and statements can be broken.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 MSG PIC X(70) VALUE "A literal that fills the line up to colum
      -    "n 72 and goes on and on.".
       PROCEDURE DIVISION.
           DISPLAY MSG.
           STOP RUN.

$ ./demo
A literal that fills the line up to column 72 and goes on and on.
```

Figure 12. A continued literal

Comments and debugging lines

A line with `*` or `/` in column 7 is a comment. The floating comment `*>` that starts a comment in the middle of a line is recognized only in free format. A debugging line, with `D` in column 7, is treated as a comment unless the program is translated with `-debug`; this is how test displays are kept in a program without compiling them in the production version.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       PROCEDURE DIVISION.
           DISPLAY "begin".
      D    DISPLAY "debugging: halfway".
           DISPLAY "end".
           STOP RUN.

$ cobol -debug -c demo.cob -o demo
$ ./demo
begin
debugging: halfway
end
```

Figure 13. A debugging line

The 72-column trap

If a line is longer than 72 columns the translator silently drops the excess, and what is left is read as program text. The translator then reports an error somewhere near the cut, which is rarely where the mistake is. The next example moves a long name to another; the end of the line has been cut off at column 72, so that the name of the target is cut in two and the translator complains about a name that is not in the program.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 ACCOUNT-BALANCE-BEFORE-INTEREST PIC 9(5).
       01 NEW-BALANCE-AFTER-INTEREST PIC 9(5).
       PROCEDURE DIVISION.
           DISPLAY "COMPUTING".
           MOVE ACCOUNT-BALANCE-BEFORE-INTEREST TO NEW-BALANCE-AFTER-INTEREST.
           STOP RUN.

$ cobol -c demo.cob -o demo
cobol: demo.cob: line 9: unknown data name NEW-BALANCE-AFTER-INT
```

Figure 14. A line that is too long

Free format

In free format a line may begin in any column and be of any length. A comment begins with `*>` and runs to the end of the line; it may follow program text. A literal must be on one line. A statement may be spread over as many lines as you like. The sentence structure and the divisions are the same as in fixed format. The directive `>>SOURCE FORMAT IS FREE` switches to free format from the line after it, which is how the file `examples/freefmt.cob` is translated with a plain `cobol -c freefmt.cob`.

```
>>SOURCE FORMAT IS FREE
*> Free-format source: no columns, comments start with *>.
IDENTIFICATION DIVISION.
PROGRAM-ID. WORDS.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 SENTENCE   PIC X(60) VALUE "the quick brown fox jumps over the lazy dog".
01 WORD       PIC X(12).
01 PTR        PIC 99 VALUE 1.
01 N-WORDS    PIC 99 VALUE 0.
01 N-LONG     PIC 99 VALUE 0.
01 LONGEST    PIC X(12) VALUE SPACES.
01 WORD-LEN   PIC 99.
01 MAX-LEN    PIC 99 VALUE 0.
01 CAPITAL    PIC X(60).

PROCEDURE DIVISION.
main-para.
    PERFORM UNTIL PTR > FUNCTION LENGTH(FUNCTION TRIM(SENTENCE))
        MOVE SPACES TO WORD
        UNSTRING SENTENCE DELIMITED BY ALL SPACE
            INTO WORD
            WITH POINTER PTR
        END-UNSTRING
        ADD 1 TO N-WORDS
        COMPUTE WORD-LEN = FUNCTION LENGTH(FUNCTION TRIM(WORD))
        IF WORD-LEN > 4 ADD 1 TO N-LONG END-IF
        IF WORD-LEN > MAX-LEN
            MOVE WORD-LEN TO MAX-LEN
            MOVE WORD TO LONGEST
        END-IF
    END-PERFORM
    DISPLAY N-WORDS " words, " N-LONG " longer than 4 letters"
    DISPLAY "longest word: " FUNCTION TRIM(LONGEST)
    MOVE FUNCTION UPPER-CASE(SENTENCE(1:1)) TO CAPITAL(1:1)
    MOVE SENTENCE(2:) TO CAPITAL(2:)
    DISPLAY FUNCTION TRIM(CAPITAL) "."
    STOP RUN.

$ ./freefmt
09 words, 03 longer than 4 letters
longest word: quick
The quick brown fox jumps over the lazy dog.
```

Figure 15. A free-format program

Words, literals and separators

A word is made of letters, digits, hyphens and underscores. A comma or a semicolon followed by a blank is a separator, like a blank. The translator folds words to upper case, so names are not case-sensitive, but the contents of literals are kept. A name must not be one of the reserved words (Appendix A).

Form	Meaning
`"text"`, `'text'`	A character literal in quotes or apostrophes. A quote inside it is written twice. Two literals joined by & form one literal.
`X"41424344"`, `H"..."`	A hexadecimal literal: the bytes of the given hexadecimal digits.
`Z"text"`	A character literal with a null byte at its end, for passing to C.
`N"text"`, `NX"..."`	A national literal, two bytes per character.
`B"10100101"`, `BX"..."`	A boolean literal, a string of bits written as the characters 0 and 1, for items with `PIC 1`.
`12`, `-3.5`, `+0.25`	Numeric literals. The decimal point is a period, or a comma if the program says `DECIMAL-POINT IS COMMA`.
`1.5E3`, `2E-2`	A floating-point literal, converted to a decimal literal with up to 15 decimals when the program is translated.
`ZERO`, `SPACE`, `HIGH-VALUE`, `LOW-VALUE`, `QUOTE`, `NULL`, `ALL "ab"`	Figurative constants. The plural forms (ZEROS, ZEROES, SPACES, HIGH-VALUES, LOW-VALUES, QUOTES, NULLS) are the same. ALL followed by a literal repeats the literal as often as needed.

Figure 16. Literals

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       PROCEDURE DIVISION.
           DISPLAY "it's" ' "quoted"' " " "a""b".
           DISPLAY X"41424344" " " "AB" & "CD" " " 1.5E2.
           DISPLAY ALL "-=" " " B"10100101" " " N"xy".
           STOP RUN.

$ ./demo
it's "quoted" a"b
ABCD ABCD 150
-= 10100101 xy
```

Figure 17. Literals

The COPY statement

`COPY name [REPLACING ...].` is replaced by the text of the copybook `name`. The translator looks for the copybook in the directory of the file that contains the COPY statement, in the current directory, and in the directories named by `-I`, in this order. In each directory it tries the name as written, in lower case and in upper case, with no file type and with the types `cpy`, `cbl`, `cob` and `copy` (also in upper case). `COPY name OF library` and `IN library` are accepted and the library name is ignored. A copybook can contain COPY statements, nested up to 20 deep.

REPLACING changes words of the copybook while it is copied. `==text== BY ==text==` replaces a sequence of words by another; a single word can be given without the equals signs. `LEADING` and `TRAILING` replace the beginning or the end of a longer word, which is how a prefix such as `CUST-` can be changed. The comparison is made on whole words of the source, so a word that is part of a longer word, such as `:TAG:-ID`, cannot be replaced except with LEADING or TRAILING.

```
      * cust.cpy
       01 CUST-RECORD.
          05 CUST-ID    PIC 9(4).
          05 CUST-NAME  PIC X(10).

      * demo.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       COPY CUST REPLACING LEADING ==CUST== BY ==OLD==.
       COPY CUST REPLACING ==CUST-RECORD== BY ==NEW-REC==
           ==CUST-ID== BY ==NEW-ID==
           ==CUST-NAME== BY ==NEW-NAME==.
       REPLACE ==MAX-N== BY ==100==.
       01 LIMIT-V PIC 999 VALUE MAX-N.
       REPLACE OFF.
       PROCEDURE DIVISION.
           MOVE 7 TO OLD-ID.
           MOVE "ABC" TO NEW-NAME.
           DISPLAY OLD-ID " " NEW-NAME " " LIMIT-V.
           STOP RUN.

$ ./demo
0007 ABC        100
```

Figure 18. COPY with REPLACING, and REPLACE

`REPLACE ==a== BY ==b== ... .` has the same effect on the text that follows it, up to `REPLACE OFF.` or the next REPLACE statement, whatever the file it comes from. The statement must start a sentence, which means that the preceding text must end with a period.

Compiler directives

A line whose first non-blank characters are >> is a directive, in fixed and in free format. The directives are described in the next table. A directive line is not part of the program text, and the text inside an inactive `>>IF` branch is skipped completely, so it need not even be valid COBOL.

Directive	Meaning
`>>SOURCE FORMAT IS FREE`	Reads the lines after this one in free format. `FIXED` goes back to fixed format.
`>>DEFINE name [AS] value`	Defines name. `>>DEFINE CONSTANT name AS value` is the same. A name defined without a value has the value 1. The option `-Dname=value` does it from the command line.
`>>IF name DEFINED`	The following lines are read if name is defined. `NOT` in front of the name reverses the test.
`>>IF name = value`	The test is true if name has this value. A name alone is true if its value is not empty, `0` or `FALSE`.
`>>ELIF condition`, `>>ELSE`	The next alternative of the `>>IF`. A condition that is not needed is not evaluated.
`>>END-IF`	Ends the conditional text. The directives can be nested up to 32 deep.

Figure 19. Directives

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 TOTAL PIC 9(4) VALUE 42.
       PROCEDURE DIVISION.
           >>IF SHOW-TOTAL DEFINED
               DISPLAY "the total is " TOTAL
           >>ELSE
               DISPLAY "the total is not shown"
           >>END-IF
           STOP RUN.

$ cobol -DSHOW-TOTAL -c demo.cob -o demo
$ ./demo
the total is 0042
```

Figure 20. Conditional compilation

Chapter 4. Program Structure

A COBOL program has up to four divisions, in a fixed order: the identification division, the environment division, the data division and the procedure division. Each division consists of sections, a section of paragraphs or entries, and the procedure division of sentences. A COBOL source file holds one or more programs, one after the other or nested.

The divisions

Division	Contents
`IDENTIFICATION DIVISION.`	`PROGRAM-ID. name [AS "literal"] [IS COMMON | INITIAL | RECURSIVE [PROGRAM]].` names the program. The words `IDENTIFICATION DIVISION.` may be left out, or abbreviated to `ID DIVISION.`. The comment paragraphs AUTHOR, INSTALLATION, DATE-WRITTEN, DATE-COMPILED, SECURITY and REMARKS are skipped, as is any text up to the next division; do not use the word DATA, ENVIRONMENT or PROCEDURE in them.
`ENVIRONMENT DIVISION.`	The CONFIGURATION SECTION with SPECIAL-NAMES (DECIMAL-POINT IS COMMA, CURRENCY SIGN, CLASS, SYMBOLIC CHARACTERS, and mnemonic names for implementor names) and the REPOSITORY; the INPUT-OUTPUT SECTION with the FILE-CONTROL paragraph that holds the SELECT entries (Chapter 10). SOURCE-COMPUTER, OBJECT-COMPUTER, ALPHABET and I-O-CONTROL are accepted and have no effect.
`DATA DIVISION.`	The FILE SECTION, the WORKING-STORAGE SECTION, the LOCAL-STORAGE SECTION and the LINKAGE SECTION (Chapter 5). The REPORT, SCREEN and COMMUNICATION sections are skipped.
`PROCEDURE DIVISION [USING ...] [RETURNING ...].`	The statements, in paragraphs and sections, and the DECLARATIVES (Chapters 6 and 10).

Figure 21. The divisions of a program

Special names

`DECIMAL-POINT IS COMMA` exchanges the roles of the period and the comma in numeric literals and pictures. `CURRENCY SIGN IS "c"` makes the character c the currency symbol of edited pictures instead of the dollar sign. `CLASS name IS "ABC" "0" THRU "9"` defines a class of characters, which a condition can test with `IF field IS name`. `SYMBOLIC CHARACTERS name IS 10` defines a name for the character with the given ordinal position (1 for the first character of the character set), which can be used like a literal. `implementor IS mnemonic` gives a mnemonic name to an implementor name such as SYSERR; DISPLAY ... UPON mnemonic then works (Chapter 6).

Paragraphs, sections and sentences

The procedure division is a series of paragraphs, optionally grouped into sections. A paragraph begins with its name, which starts in area A and is followed by a period; the sentences of the paragraph follow. A section begins with its name, the word SECTION and a period, and holds paragraphs, or sentences directly. A **sentence** is one or more statements ended by a period.

Execution begins at the first statement of the procedure division and continues in the order of the text, from a paragraph into the next one. Control is transferred with GO TO, PERFORM, CALL and the statements that end a program. `PERFORM name` runs a paragraph or a section and returns to the statement after the PERFORM; `PERFORM first THRU last` runs the paragraphs from first to last. The end of a performed paragraph or section is the end of its text, so a paragraph that is performed must not be left with a GO TO to a paragraph outside the range, unless the program never returns.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
```

```
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 N PIC 9 VALUE 2.
       PROCEDURE DIVISION.
       MAIN-PARA.
           DISPLAY "start".
           PERFORM STEP-A.
           PERFORM STEP-B THRU STEP-C.
           PERFORM SEC-ONE.
           GO TO PICK-1 PICK-2 PICK-3 DEPENDING ON N.
           DISPLAY "not reached when N is 1 to 3".
       PICK-1.
           DISPLAY "picked 1".
           GO TO FINISH-UP.
       PICK-2.
           DISPLAY "picked 2".
           GO TO FINISH-UP.
       PICK-3.
           DISPLAY "picked 3".
       FINISH-UP.
           DISPLAY "stop".
           STOP RUN.
       STEP-A.
           DISPLAY "step A".
       STEP-B.
           DISPLAY "step B".
       STEP-C.
           DISPLAY "step C".
       SEC-ONE SECTION.
       SEC-PARA-1.
           DISPLAY "section, first paragraph".
           IF N = 2 EXIT SECTION END-IF.
           DISPLAY "section, not shown when N is 2".
       SEC-PARA-2.
           DISPLAY "section, second paragraph".

$ ./demo
start
step A
step B
step C
section, first paragraph
picked 2
stop
```

Figure 22. Paragraphs, sections and the ways out of them

`EXIT PARAGRAPH` leaves the paragraph that contains it, and `EXIT SECTION` the section; both are errors outside a paragraph or a section. `GO TO name ... DEPENDING ON n` jumps to the nth name, and goes on with the next statement if n is outside the range. A `GO TO` without a name is accepted and stops the program with a run-time error if it is executed. The statement `ALTER` is not implemented.

Ending a program

`STOP RUN` ends the program, closing all open files, and gives the value of RETURN-CODE as the exit status of the process. `STOP RUN WITH ERROR STATUS n` and `WITH NORMAL STATUS n` set RETURN-CODE to n first. `GOBACK` and `EXIT PROGRAM` return from a called program to its caller; in the main program GOBACK ends the program. `STOP literal` is accepted and treated as `STOP RUN`.

Programs in one file

A source file can contain any number of programs. The first program that is not nested and not a function is the main program; the run begins there. All the other programs of the translation unit (the files of one `cobol` command) are known by name and can be called with CALL from anywhere. A program may be written after the end of another one, or it can be **nested**: it is written inside another program, after its procedure division and before its `END PROGRAM`. A nested program can use the items of its parent that are declared GLOBAL. In this release every program, nested or not, is also callable by name from every other, so the keyword COMMON is accepted and has no further effect.

`END PROGRAM name.` ends a program. It is needed to end the outer program when programs are nested, and recommended always.

Attribute	Meaning
(none)	The data of the program is initialized the first time it is called. It keeps its values from one call to the next.
`INITIAL`	The WORKING-STORAGE of the program is initialized again at every call, as if the program had been cancelled.
`COMMON`	Accepted; see above.
`RECURSIVE`	Accepted. A program that calls itself keeps the data of its levels in LOCAL-STORAGE, but only one copy of it exists in this release, so recursion that needs local data does not work (Chapter 15). Use PERFORM for recursion in the same program, or keep the recursion depth in a table.

Figure 23. Program attributes

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. OUTER.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 SHARED PIC 99 VALUE 10 GLOBAL.
       PROCEDURE DIVISION.
           CALL "COUNTER".
           CALL "COUNTER".
           CALL "FRESH".
           CALL "FRESH".
           CALL "SHOWIT".
           STOP RUN.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. COUNTER.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 C PIC 99 VALUE 0.
       PROCEDURE DIVISION.
           ADD 1 TO C.
           DISPLAY "COUNTER " C.
           EXIT PROGRAM.
       END PROGRAM COUNTER.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. FRESH IS INITIAL.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 C PIC 99 VALUE 0.
       PROCEDURE DIVISION.
           ADD 1 TO C.
           DISPLAY "FRESH " C.
           EXIT PROGRAM.
       END PROGRAM FRESH.
```

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. SHOWIT.
       PROCEDURE DIVISION.
           DISPLAY "shared = " SHARED.
           EXIT PROGRAM.
       END PROGRAM SHOWIT.
       END PROGRAM OUTER.

$ ./demo
COUNTER 01
COUNTER 02
FRESH 01
FRESH 01
shared = 10
```

Figure 24. Nested programs, INITIAL and GLOBAL

Special registers

A few names are known without a declaration. `RETURN-CODE` is a binary field that CALL, STOP RUN and the exit status use. `TALLY` is a nine-digit binary item for INSPECT, `SORT-RETURN` is declared for compatibility, and `LINAGE-COUNTER` is declared but not maintained. `ADDRESS OF item`, `LENGTH OF item` and `NUMBER-OF-CALL-PARAMETERS` can be used where an operand is allowed, and `NULL` is the null pointer. `SELF` and `SUPER` are available in methods (Chapter 13).

Chapter 5. The Data Division

The data division describes all the data of the program. In the WORKING-STORAGE SECTION it describes the data that exists as long as the program does. The LOCAL-STORAGE SECTION describes data that is created anew at every call. The LINKAGE SECTION describes data that belongs to the caller, and the FILE SECTION the records of files (Chapter 10).

Data descriptions

A data description entry is a level number, a name, and clauses, ended by a period. A level number of 01 or 77 begins a new data item that is not part of another one; the numbers 02 to 49 describe the parts of a group. An item with parts is a **group item**; its value is the characters of its parts, and it is treated as an alphanumeric field. An item without parts is an **elementary item** and has a PICTURE or a usage that decides its type. The name FILLER, or no name, stands for a part that the program does not use.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 EMPLOYEE.
          05 EMP-NAME.
             10 FIRST-NAME   PIC X(8)     VALUE "ADA".
             10 LAST-NAME    PIC X(10)    VALUE "LOVELACE".
          05 EMP-PAY.
             10 SALARY       PIC 9(6)V99  VALUE 52000.50.
             10 BONUS        PIC 9(4).
          05 EMP-FLAGS       PIC XXX      VALUE ALL "N".
       77 STEPS              PIC S9(3)    VALUE -12.
       PROCEDURE DIVISION.
           DISPLAY EMPLOYEE.
           DISPLAY LAST-NAME OF EMP-NAME " in "
               LENGTH OF EMPLOYEE " bytes".
           DISPLAY SALARY " " BONUS " " STEPS.
           MOVE "GRACE" TO FIRST-NAME.
           DISPLAY EMP-NAME.
           STOP RUN.

$ ./demo
ADA     LOVELACE  052000500000NNN
LOVELACE   in 33 bytes
052000.50 0000 -012
GRACE   LOVELACE
```

Figure 25. Levels, groups and initial values

An item that has no VALUE clause starts as zero if it is numeric and as blanks if it is alphanumeric; a pointer starts as the null pointer. A group without VALUE clauses in its parts starts as blanks, so a group with numeric parts has each of them set to zero. DISPLAY of a group writes its bytes unchanged, as in the first line above; DISPLAY of a numeric item writes its sign and decimal point, as in the third line (Chapter 6).

A name can be made unique by qualification: `LAST-NAME OF EMP-NAME`, or `IN`, naming the groups from the inner to the outer. A reference to a name that is declared more than once without qualification is an error. Names of other programs' data are not visible, except the GLOBAL items of the enclosing programs.

The PICTURE clause

`PIC` or `PICTURE [IS] string` gives the type of an elementary item and its size. A character may be repeated by a count in parentheses: `9(5)` is `99999`.

Symbol	Meaning
`9`	A decimal digit.
`X`	Any character. `A` is a letter or a space; the item is alphanumeric in both cases. `N` is a national character of two bytes.
`1`	A bit; an item of `1`s is a boolean item that is written as a string of 0 and 1 characters, one per bit position, and given values such as `B"10100101"`.
`S`	The item is signed. It takes no space.
`V`	The implied decimal point. It takes no space.
`P`	A scaling position: the number is a multiple of a power of ten. `PIC 999PP` holds 12300 in three digits 123; `PIC PP99` holds 0.0012 as 12.
`Z`	A digit position of an edited item; a leading zero is replaced by a blank.
`*`	As `Z`, but leading zeros are replaced by asterisks.
`+ -`	The sign of an edited item. One at the end or at the beginning is a fixed sign (`-` shows a minus sign for negative values and a blank for positive ones, `+` shows both). Repeated, they are a floating sign that stands directly before the first non-zero digit.
`CR DB`	At the end of an edited item: the letters shown for a negative value. A blank pair is shown for other values.
`$`	The currency sign, fixed or floating like `+` and `-`. The sign can be changed by CURRENCY SIGN.
`, .`	The comma and the decimal point of an edited item. With DECIMAL-POINT IS COMMA they are exchanged.
`B 0 /`	A blank, a zero or a slash inserted at this place in an edited item.

Figure 26. Picture characters

The category of an item follows from its picture. A picture of only `9`, with an optional `S`, `V` or `P`, is a **numeric** item. A picture with `X`, `A`, or a mixture of `9` with these is **alphanumeric**; if it also has `B`, `0` or `/` it is **alphanumeric edited**. A picture with `Z`, `*`, `+`, `-`, `$`, `,`, `.`, `CR`, `DB`, `B`, `0` or `/` together with digit positions is **numeric edited**: its value is a string for printing, which can be moved to a numeric item again. A numeric item has at most 80 digits (Chapter 15).

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 AMOUNT   PIC S9(5)V99 VALUE -1234.5.
       01 E1       PIC ZZ,ZZ9.99.
       01 E2       PIC $$$,$$9.99.
       01 E3       PIC -(5)9.99.
       01 E4       PIC ZZ,ZZ9.99CR.
       01 E5       PIC **,**9.99.
       01 E6       PIC 99/99/9999.
       01 E7       PIC 999B999B999.
       01 E8       PIC +9(4).99.
       01 BACK     PIC S9(5)V99.
       PROCEDURE DIVISION.
           MOVE AMOUNT TO E1 DISPLAY "[" E1 "]".
           MOVE AMOUNT TO E2 DISPLAY "[" E2 "]".
           MOVE AMOUNT TO E3 DISPLAY "[" E3 "]".
           MOVE AMOUNT TO E4 DISPLAY "[" E4 "]".
           MOVE 5.5 TO E5 DISPLAY "[" E5 "]".
           MOVE 12312026 TO E6 DISPLAY "[" E6 "]".
           MOVE 123456789 TO E7 DISPLAY "[" E7 "]".
           MOVE 12.5 TO E8 DISPLAY "[" E8 "]".
           MOVE E4 TO BACK DISPLAY BACK.
           STOP RUN.

$ ./demo
[ 1,234.50]
[ $1,234.50]
[ -1234.50]
[ 1,234.50CR]
[*****5.50]
[12/31/2026]
[123 456 789]
[+0012.50]
-01234.50
```

Figure 27. Numeric edited pictures

The USAGE clause

The USAGE clause says how a number is stored. `USAGE IS` can be left out, and the usage can also be written alone: `PIC 9(4) COMP`. The usage of a group is inherited by its parts. COBOL has many names for the binary and floating usages; the next table lists those that are accepted and the storage they use.

Usage	Size	Representation
`DISPLAY`	One byte per digit, plus one for a separate sign	Characters. The default. A sign without `SEPARATE` is joined to the last digit (to the first with `SIGN LEADING`): for a negative number the digit byte has the code of the digit plus 0x40 in ASCII, so that `-123` is stored as `12s`; on an EBCDIC machine the zone D is used for negative and C for positive numbers.
`COMP`, `COMPUTATIONAL`, `COMP-4`, `BINARY`	2 bytes for 1 to 4 digits, 4 bytes for 5 to 9, 8 bytes for 10 to 18, 16 bytes for 19 to 38	Binary integer, big-endian, two's complement if the picture has an S. The value is cut to the digits of the picture.
`COMP-5`, `COMP-X`	As COMP	Binary integer in the byte order of the machine. The value is cut to the digits of the picture.
`COMP-3`, `PACKED-DECIMAL`	Digits / 2 + 1 (rounded down)	Two digits per byte; the last half byte is the sign: C or D, or F if there is no S in the picture.

Usage	Size	Representation
COMP-1, FLOAT-SHORT	4	C `float`. No PICTURE.
COMP-2, FLOAT-LONG, FLOAT-EXTENDED	8	C `double`. No PICTURE.
BINARY-CHAR, BINARY-SHORT, BINARY-LONG, BINARY-DOUBLE	1, 2, 4, 8	Native binary integer, signed unless the word UNSIGNED follows. No PICTURE. The value wraps around at the size of the type; it is not cut to a number of digits.
INDEX	4	A binary table position; also made by INDEXED BY.
POINTER, FUNCTION-POINTER, PROCEDURE-POINTER	8	The address of a data item. `POINTER TO name` is accepted.
OBJECT REFERENCE [class]	8	A reference to an object (Chapter 13).
NATIONAL	2 per character	National (two-byte) characters. `PIC N` has this usage.

Figure 28. Usages and their size in bytes

The option `-binsize 1-2-4-8` gives a binary item of one or two digits one byte. The usages are shown in the next example, which also shows what DISPLAY writes.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 U-DISP    PIC S9(5)V99 VALUE -123.45.
       01 U-SEP     PIC S9(5)    SIGN LEADING SEPARATE VALUE -42.
       01 U-COMP    PIC S9(4)    COMP VALUE 1234.
       01 U-COMP8   PIC S9(12)   COMP VALUE 1.
       01 U-PACK    PIC S9(5)V99 COMP-3 VALUE -123.45.
       01 U-FLT     COMP-1 VALUE 1.5.
       01 U-DBL     COMP-2 VALUE 2.25.
       01 U-LONG    BINARY-LONG VALUE 100000.
       01 U-CHAR    BINARY-CHAR UNSIGNED VALUE 255.
       01 U-NAT     PIC N(3) VALUE N"abc".
       PROCEDURE DIVISION.
           DISPLAY LENGTH OF U-DISP " " LENGTH OF U-SEP " "
               LENGTH OF U-COMP " " LENGTH OF U-COMP8 " "
               LENGTH OF U-PACK " " LENGTH OF U-FLT " "
               LENGTH OF U-DBL " " LENGTH OF U-LONG " "
               LENGTH OF U-CHAR " " LENGTH OF U-NAT.
           DISPLAY U-DISP " " U-SEP " " U-COMP " " U-PACK.
           DISPLAY U-FLT " " U-DBL " " U-LONG " " U-CHAR " " U-NAT.
           ADD 1 TO U-CHAR.
           DISPLAY U-CHAR.
           STOP RUN.

$ ./demo
7 6 2 8 4 4 8 4 1 6
-00123.45 -00042 +1234 -00123.45
1.5 2.25 +0000100000 255 abc
000
```

Figure 29. Usages and sizes

VALUE and REDEFINES

`VALUE [IS] literal` gives the initial value of an item. The literal must suit the item: a numeric literal for a numeric item, a character literal or a figurative constant for the others. `VALUE ALL "ab"` repeats the string;

`VALUE ZERO`, `SPACE`, `HIGH-VALUE`, `LOW-VALUE` and `QUOTE` are the figurative constants. A VALUE clause on a table item sets every occurrence. A VALUE clause on a group item is applied as a MOVE of an alphanumeric literal to the whole group, so it overrides the clauses of the parts.

`REDEFINES name` makes an item occupy the storage of another item at the same level, which has to come directly before it or before another item that redefines it. The item can be of another type and size; the storage of the group is the size of the longest. At level 01 a record may redefine another record. The level 66 entry `RENAMES` is parsed, but the renamed item cannot be used in this release (Chapter 15).

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 DATE-REC.
          05 YEAR-PART    PIC 9(4).
          05 MONTH-PART   PIC 99.
          05 DAY-PART     PIC 99.
       01 DATE-NUM REDEFINES DATE-REC PIC 9(8).
       01 WORK-AREA.
          05 BY-NAME      PIC X(8)  VALUE "01234567".
          05 BY-TABLE REDEFINES BY-NAME.
             10 CHARS     PIC X OCCURS 8 TIMES.
       PROCEDURE DIVISION.
           MOVE 20261012 TO DATE-NUM.
           DISPLAY YEAR-PART "-" MONTH-PART "-" DAY-PART.
           MOVE 3 TO MONTH-PART.
           DISPLAY DATE-NUM.
           DISPLAY CHARS (1) CHARS (8).
           STOP RUN.

$ ./demo
2026-10-12
20260312
07
```

Figure 30. REDEFINES

Tables: OCCURS

`OCCURS n [TIMES]` makes an item a table of n elements; a group may have the clause, and tables can be nested. `OCCURS min TO max [TIMES] DEPENDING ON name` makes the table variable: the current number of elements is the value of name (Chapter 8). `ASCENDING KEY IS k ...` and `DESCENDING KEY IS k ...` name the keys of the table for SORT; `INDEXED BY i ...` declares index items for the table. A table has up to three dimensions in the form of the clause. Subscripts, indexes, SEARCH and the sorting of tables are described in Chapter 8.

Condition names

A level 88 entry gives a name to a value, a list of values, or a range of values of the item before it. The name can then be used as a condition: it is true if the item has one of the values. `SET name TO TRUE` moves the first value to the item, and `WHEN SET TO FALSE IS value` names the value that `SET name TO FALSE` moves.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 GRADE        PIC 9(3).
          88 PASSING    VALUE 50 THRU 100.
          88 PERFECT    VALUE 100.
          88 BAD-DATA   VALUE 0 101 THRU 999.
       01 SWITCH       PIC X VALUE "N".
          88 ON-STATE  VALUE "Y" WHEN SET TO FALSE IS "N".
       PROCEDURE DIVISION.
           MOVE 75 TO GRADE.
           IF PASSING DISPLAY "passing" END-IF.
           IF NOT PERFECT DISPLAY "not perfect" END-IF.
           MOVE 101 TO GRADE.
           IF BAD-DATA DISPLAY "bad data" END-IF.
           SET ON-STATE TO TRUE.
           DISPLAY SWITCH.
           SET ON-STATE TO FALSE.
           DISPLAY SWITCH.
           SET PERFECT TO TRUE.
           DISPLAY GRADE.
           STOP RUN.

$ ./demo
passing
not perfect
bad data
Y
N
100
```

Figure 31. Condition names

Constants

`01 name CONSTANT [AS] value.` and `78 name VALUE value.` define a name for a value. The name is not stored; the value is used wherever the name is written, in a procedure division expression as well as in a VALUE clause. Use `LENGTH OF item` in the program for the size of an item; the form `CONSTANT AS LENGTH OF item` is accepted but gives zero in this release. A constant cannot be used as the count of an OCCURS clause.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       78 MAX-ITEMS VALUE 5.
       01 TAX-RATE CONSTANT AS 0.2.
       01 TITLE     CONSTANT "Order".
       01 TABLE-1.
          05 ITEM-PRICE PIC 9(3)V99 OCCURS 5 TIMES.
       01 TOTAL PIC 9(5)V99.
       PROCEDURE DIVISION.
           MOVE 50 TO ITEM-PRICE (1).
           COMPUTE TOTAL = ITEM-PRICE (1) * (1 + TAX-RATE).
           DISPLAY TITLE " " TOTAL " " MAX-ITEMS.
           STOP RUN.

$ ./demo
Order 00060.00 5
```

Figure 32. Constants

Other clauses

Clause	Meaning
`JUSTIFIED RIGHT`, `JUST`	For an alphanumeric item: a value that is moved to it is aligned at the right, and the left end is cut if it is too long. Without it, the value is aligned at the left and padded with blanks.
`BLANK WHEN ZERO`	For a numeric edited item: the item is all blanks when its value is zero.
`SIGN [IS] LEADING | TRAILING [SEPARATE [CHARACTER]]`	The place of the sign of a signed DISPLAY item: joined to the first or last digit, or a separate character + or - before or after the digits. The size grows by one with SEPARATE.
`SYNCHRONIZED`, `SYNC [LEFT | RIGHT]`	A binary item is placed at a position in its record that is a multiple of its size (2, 4 or 8), counting from the start of the 01 item. Padding bytes are put in front of it.
`GLOBAL`	The item can be used by the programs nested in the program that declares it.
`EXTERNAL`	Accepted; the item is private to the program in this release.
`BASED`	The item has no storage until ALLOCATE or SET ADDRESS OF gives it some (Chapter 6).
`SIGNED`, `UNSIGNED`, `ALIGNED`, `DYNAMIC`	Accepted and ignored.

Figure 33. Further clauses of a data description

The clauses `TYPEDEF` and `TYPE` are not implemented, nor is the `PROPERTY` clause.

The LINKAGE SECTION and the other sections

Items of the LINKAGE SECTION have no storage of their own: they describe arguments or results, and are bound to the storage of the caller when the program is called with USING or RETURNING (Chapter 12). LOCAL-STORAGE items are created and initialized at each call of the program and deleted when it returns; WORKING-STORAGE items are initialized once, when the program is first called, unless the program is INITIAL.

Chapter 6. Procedure Division Statements

The procedure division holds the statements of the program. A **sentence** is one or more statements ended by a period. Most statements can be ended by a scope terminator, `END-IF`, `END-PERFORM` and so on, which lets a statement be nested in another one without a period; the examples in this manual use the terminators and put a period only at the end of a paragraph. A period ends all open statements.

Group	Statements
Data movement	MOVE, INITIALIZE, SET, STRING, UNSTRING, INSPECT
Arithmetic	ADD, SUBTRACT, MULTIPLY, DIVIDE, COMPUTE
Control	IF, EVALUATE, PERFORM, GO TO, EXIT, CONTINUE, NEXT SENTENCE, STOP, GOBACK
Input and output	ACCEPT, DISPLAY, OPEN, CLOSE, READ, WRITE, REWRITE, DELETE, START
Tables	SEARCH, SEARCH ALL, SORT, MERGE, RELEASE, RETURN
Subprograms	CALL, CANCEL, INVOKE, ALLOCATE, FREE

Figure 34. The statements

DISPLAY and ACCEPT

`DISPLAY item ...` writes the items one after the other and ends the line. Literals and fields can be mixed. `UPON SYSERR` or `UPON STDERR` writes to the standard error file, and `WITH NO ADVANCING` leaves out the line end. An alphanumeric item is written as it is. A numeric item that is not edited is written in a readable form: a sign if the item is signed, and a decimal point where the picture has a V, so that a `PIC S9(5)V99` with the value -123.45 shows as `-00123.45`. A group is written byte for byte. A COMP-1 or COMP-2 item is written with up to 15 significant digits.

`ACCEPT item` reads a line from the standard input. `ACCEPT item FROM DATE` (YYMMDD), `DATE YYYYMMDD`, `DAY` (YYDDD), `DAY YYYYDDD`, `DAY-OF-WEEK` (1 for Monday), `TIME` (HHMMSSss), `COMMAND-LINE`, `ARGUMENT-NUMBER` and `ENVIRONMENT "NAME"` get values from the system. `ARGUMENT-VALUE` returns the next argument of the command line; `DISPLAY n UPON ARGUMENT-NUMBER` selects the argument.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 NAME-IN    PIC X(20).
       01 N-ARGS     PIC 9.
       01 ARG        PIC X(10).
       01 HOME-DIR   PIC X(30).
       PROCEDURE DIVISION.
           ACCEPT NAME-IN.
           DISPLAY "Hello, " FUNCTION TRIM (NAME-IN) "!".
           ACCEPT N-ARGS FROM ARGUMENT-NUMBER.
           DISPLAY "arguments: " N-ARGS.
           ACCEPT ARG FROM ARGUMENT-VALUE.
           DISPLAY "first: " ARG.
           ACCEPT HOME-DIR FROM ENVIRONMENT "DEMOVAR".
           DISPLAY "DEMOVAR=" FUNCTION TRIM (HOME-DIR).
           DISPLAY "to stderr" UPON SYSERR.
           DISPLAY "no newline" WITH NO ADVANCING.
           DISPLAY " - done".
           STOP RUN.

$ DEMOVAR=hopper ./demo one two
Hello, Grace!
arguments: 2
first: one
DEMOVAR=hopper
to stderr
no newline - done
```

Figure 35. ACCEPT and DISPLAY

MOVE

`MOVE source TO target ...` copies a value into one or more fields, converting it to the type of each target. A numeric value is aligned on the decimal point and cut or padded with zeros at both ends; an alphanumeric value is aligned at the left (at the right with JUSTIFIED RIGHT) and padded with blanks; a move to an edited item formats it. A move from a numeric item to an alphanumeric one writes the digits; from an alphanumeric item to a numeric one reads them. `MOVE CORRESPONDING a TO b` (or `CORR`) moves the parts of a that have a part of the same name in b. Reference modification, `item (start : length)`, selects part of an alphanumeric item; the length can be left out.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 SRC.
          05 ID-NO    PIC 9(4)    VALUE 1007.
          05 NAME     PIC X(6)    VALUE "ADA".
          05 PAY      PIC 9(4)V9  VALUE 250.5.
       01 DST.
          05 NAME     PIC X(4).
          05 PAY      PIC 9(6)V99.
          05 ID-NO    PIC X(6).
       01 SMALL       PIC 99.
       01 TXT         PIC X(12) VALUE "ABCDEFGHIJKL".
       01 RJ          PIC X(5) JUSTIFIED RIGHT.
       PROCEDURE DIVISION.
           MOVE CORRESPONDING SRC TO DST.
           DISPLAY DST.
           MOVE 12345 TO SMALL.
           DISPLAY SMALL.
           MOVE ID-NO OF SRC TO ID-NO OF DST.
           DISPLAY "[" ID-NO OF DST "]".
           DISPLAY TXT (4:3) "," TXT (10:).
           MOVE "xy" TO TXT (2:2).
           MOVE "ab" TO RJ.
           DISPLAY TXT "[" RJ "]".
           MOVE ALL "-" TO TXT.
           DISPLAY TXT.
           STOP RUN.

$ ./demo
ADA 000250501007
45
[1007  ]
DEF,JKL
AxyDEFGHIJKL[   ab]
------------
```

Figure 36. MOVE and reference modification

Arithmetic

`ADD a ... TO b ...`, `ADD a ... TO b GIVING c ...`, `SUBTRACT a ... FROM b ...`, `MULTIPLY a BY b [GIVING c]`, `DIVIDE a INTO b`, `DIVIDE a BY b GIVING c [REMAINDER r]` and `COMPUTE c = expression` store a result in one or more fields. Every operation is exact. The result is aligned on the decimal point of the target and truncated, unless ROUNDED is written after the target; the operands may be literals, numeric fields, numeric edited fields and intrinsic functions. In `DIVIDE ... REMAINDER` the remainder is the dividend minus the product of the truncated quotient and the divisor.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 A        PIC 9(3)V99 VALUE 10.25.
       01 B        PIC 9(3)V99 VALUE 3.
       01 R        PIC 9(4)V99.
       01 Q        PIC 99.
       01 REM       PIC 99V99.
       01 N        PIC S9(3) VALUE 10.
       PROCEDURE DIVISION.
           ADD A TO B GIVING R.            DISPLAY "add      " R.
           SUBTRACT 1 FROM R.              DISPLAY "subtract " R.
           MULTIPLY A BY B GIVING R.       DISPLAY "multiply " R.
           DIVIDE A BY B GIVING Q REMAINDER REM.
           DISPLAY "divide   " Q " rem " REM.
           COMPUTE R = A / B.              DISPLAY "compute  " R.
           COMPUTE R ROUNDED = A / B.      DISPLAY "rounded  " R.
           COMPUTE R = (A + B) ** 2 - 2 * B.
           DISPLAY "formula  " R.
           ADD 1 TO N, N.                  DISPLAY "twice    " N.
           STOP RUN.

$ ./demo
add      0013.25
subtract 0012.25
multiply 0030.75
divide   03 rem 01.25
compute  0003.41
rounded  0003.42
formula  0169.56
twice    +012
```

Figure 37. Arithmetic statements

ROUNDED and SIZE ERROR

`ROUNDED` rounds the result to the places of the target instead of cutting it off; the usual rule, away from zero for a half, is the default. `ROUNDED MODE IS mode` selects another one. The mode names of the standard are accepted:

Mode	Effect
`NEAREST-AWAY-FROM-ZERO`	To the nearest value; a half goes away from zero. This is the default of ROUNDED.
`NEAREST-EVEN`	To the nearest value; a half goes to the even neighbour.
`NEAREST-TOWARD-ZERO`	To the nearest value; a half goes toward zero.
`AWAY-FROM-ZERO`	Any non-zero rest rounds the magnitude up.
`TRUNCATION`	The rest is cut off, as without ROUNDED.
`TOWARD-GREATER`, `TOWARD-LESSER`	Accepted. In this release they round to the nearest value and send a half toward plus or minus infinity; they do not yet round a non-half rest in a direction (Chapter 15).
`PROHIBITED`	Accepted, no rounding.

Figure 38. Rounding modes

`ON SIZE ERROR` is performed when the result does not fit into the target, and when a division by zero or an invalid exponentiation occurs. In that case the target keeps its old value. `NOT ON SIZE ERROR` is performed otherwise. Without a SIZE ERROR phrase a result that is too large is stored with its high-order digits cut off, as the standard allows.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 X        PIC 9V99 VALUE 2.50.
       01 R        PIC 9.
       01 SMALL    PIC 99 VALUE 95.
       01 ZERO-VAL PIC 9 VALUE 0.
       01 Z        PIC 99 VALUE 77.
       PROCEDURE DIVISION.
           COMPUTE R ROUNDED = X.            DISPLAY "default  " R.
           COMPUTE R ROUNDED MODE IS NEAREST-EVEN = X.
           DISPLAY "even     " R.
           COMPUTE R ROUNDED MODE IS NEAREST-TOWARD-ZERO = X.
           DISPLAY "toward 0 " R.
           COMPUTE R ROUNDED MODE IS AWAY-FROM-ZERO = 2.01.
           DISPLAY "away     " R.
           COMPUTE R ROUNDED MODE IS TRUNCATION = 2.99.
           DISPLAY "truncate " R.
           ADD 10 TO SMALL
               ON SIZE ERROR DISPLAY "size error, SMALL = " SMALL
               NOT ON SIZE ERROR DISPLAY "fits"
           END-ADD.
           DIVIDE Z BY ZERO-VAL GIVING Z
               ON SIZE ERROR DISPLAY "division by zero, Z = " Z
           END-DIVIDE.
           ADD 10 TO SMALL.
           DISPLAY "cut off  " SMALL.
           STOP RUN.

$ ./demo
default  3
even     2
toward 0 2
away     3
truncate 2
size error, SMALL = 95
division by zero, Z = 77
cut off  05
```

Figure 39. Rounding and size errors

Choosing: IF and EVALUATE

`IF condition THEN statements [ELSE statements] END-IF` performs the first group when the condition is true. `NEXT SENTENCE` jumps to the statement after the next period; `CONTINUE` does nothing. Conditions are described in Chapter 7.

`EVALUATE` is the multi-way choice of COBOL. It names one or more **subjects**, which are expressions or the word TRUE or FALSE, and a list of `WHEN` groups; the first group whose **objects** match the subjects is performed. A WHEN object is a value, a range `a THRU b`, a condition, `ANY`, or, for the subject TRUE, a condition. Several subjects are joined by `ALSO`, and then every WHEN group has one object per subject. `WHEN OTHER` matches everything else.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 SCORE    PIC 999.
       01 KIND     PIC X.
       01 REGION   PIC X(5).
       PROCEDURE DIVISION.
           PERFORM VARYING SCORE FROM 40 BY 25 UNTIL SCORE > 110
               EVALUATE TRUE
                   WHEN SCORE >= 90 DISPLAY SCORE " A"
                   WHEN SCORE >= 60 DISPLAY SCORE " B"
                   WHEN OTHER       DISPLAY SCORE " fail"
               END-EVALUATE
           END-PERFORM.
           MOVE "B" TO KIND.
           MOVE "NORTH" TO REGION.
           EVALUATE KIND ALSO REGION
               WHEN "A" ALSO ANY       DISPLAY "type A anywhere"
               WHEN "B" THRU "D" ALSO "NORTH"
                                       DISPLAY "B to D in the north"
               WHEN OTHER              DISPLAY "other"
           END-EVALUATE.
           IF KIND = "A" OR "B"
               DISPLAY "A or B"
           ELSE
               DISPLAY "neither"
           END-IF.
           STOP RUN.

$ ./demo
040 fail
065 B
090 A
B to D in the north
A or B
```

Figure 40. IF and EVALUATE

PERFORM

PERFORM is the loop and the subroutine call of COBOL. The forms are:

- `PERFORM name` performs a paragraph or a section and returns. `PERFORM first THRU last` performs all paragraphs from first to last.
- `PERFORM n TIMES statements END-PERFORM` repeats the statements n times. The count may be a literal or a numeric field. The form `PERFORM name n TIMES`, with a paragraph name, is not accepted in this release; write `PERFORM n TIMES PERFORM name END-PERFORM` instead.
- `PERFORM name UNTIL condition` tests the condition before every pass. `WITH TEST AFTER` tests after the pass.
- `PERFORM name VARYING i FROM a BY b UNTIL condition` counts. `AFTER j FROM c BY d UNTIL condition` adds an inner loop; there can be several AFTER phrases.
- Each form without a name has the statements in the line, ended by `END-PERFORM`: an inline loop. `EXIT PERFORM` leaves such a loop, and `EXIT PERFORM CYCLE` jumps to its next pass.

Performs may be nested to 128 levels; a deeper nesting stops the program with a run-time error. A paragraph must not be performed again before the first perform has returned.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
```

```
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 I        PIC 99.
       01 J        PIC 99.
       01 TOTAL    PIC 999 VALUE 0.
       PROCEDURE DIVISION.
       MAIN.
           PERFORM 2 TIMES
               PERFORM SHOW
           END-PERFORM.
           PERFORM ONE THRU THREE.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > 2
               PERFORM VARYING J FROM 1 BY 1 UNTIL J > 3
                   COMPUTE TOTAL = TOTAL + I * J
               END-PERFORM
           END-PERFORM.
           DISPLAY "total " TOTAL.
           MOVE 5 TO I.
           PERFORM WITH TEST AFTER UNTIL I > 3
               DISPLAY "after-test pass, I = " I
           END-PERFORM.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > 9
               IF I = 3 EXIT PERFORM CYCLE END-IF
               IF I = 6 EXIT PERFORM END-IF
               DISPLAY "I = " I
           END-PERFORM.
           STOP RUN.
       SHOW.
           DISPLAY "show".
       ONE.
           DISPLAY "one".
       TWO.
           DISPLAY "two".
       THREE.
           DISPLAY "three".

$ ./demo
show
show
one
two
three
total 018
after-test pass, I = 05
I = 01
I = 02
I = 04
I = 05
```

Figure 41. The forms of PERFORM

GO TO and EXIT

`GO TO name` jumps to a paragraph or section. `GO TO a b c DEPENDING ON n` jumps to the n-th name, and falls through if n is out of range. `EXIT PARAGRAPH` and `EXIT SECTION` leave the paragraph or the section; `EXIT PROGRAM` and `GOBACK` return to the caller (and end the run in a main program); `STOP RUN` ends the run, and gives `RETURN-CODE` as the exit status of the process. A sentence that begins with `EXIT SECTION` is read as a section header named EXIT, so write the statement after another statement of the same sentence, as the next example does. `ALTER` and `ENTRY` are not implemented.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 CHOICE   PIC 9.
       PROCEDURE DIVISION.
       MAIN.
           PERFORM VARYING CHOICE FROM 0 BY 1 UNTIL CHOICE > 3
               PERFORM DISPATCH
           END-PERFORM.
           MOVE 3 TO RETURN-CODE.
           STOP RUN.
       DISPATCH SECTION.
       START-IT.
           GO TO FIRST-ONE SECOND-ONE DEPENDING ON CHOICE.
           DISPLAY CHOICE " is out of range"
           EXIT SECTION.
       FIRST-ONE.
           DISPLAY "first"
           EXIT SECTION.
       SECOND-ONE.
           DISPLAY "second".

$ ./demo; echo "exit status $?"
0 is out of range
first
second
3 is out of range
exit status 3
```

Figure 42. GO TO DEPENDING and EXIT

STRING, UNSTRING and INSPECT

`STRING a b ... DELIMITED BY d ... INTO t [WITH POINTER p] [ON OVERFLOW ...]` joins the sources into the target. Each source is cut at the first occurrence of its delimiter (`SIZE` takes it whole). The pointer, which starts at 1 unless the program sets it, shows where the next character goes and is updated. The rest of the target is not changed, so the program should first fill it with blanks. ON OVERFLOW is performed when the target is full.

`UNSTRING s DELIMITED BY [ALL] d [OR d2 ...] INTO t1 [DELIMITER IN x] [COUNT IN n] ... [WITH POINTER p] [TALLYING IN k] [ON OVERFLOW ...]` splits the source at the delimiters into the targets.

`INSPECT s TALLYING n FOR CHARACTERS | ALL x | LEADING x` counts, `INSPECT s REPLACING ALL | LEADING | FIRST x BY y` replaces, and `INSPECT s CONVERTING "abc" TO "xyz"` translates characters. Each phrase can be limited with `BEFORE INITIAL x` and `AFTER INITIAL x`. Up to eight phrases are allowed in one statement.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 FULL     PIC X(30) VALUE SPACES.
       01 PTR      PIC 99.
       01 LINE-IN  PIC X(40) VALUE "10,20;30,,40".
       01 PARTS.
          05 PART  PIC X(4) OCCURS 5 TIMES.
       01 DLM      PIC X.
       01 CNT      PIC 99.
       01 NPARTS   PIC 99.
       01 TXT      PIC X(20) VALUE "banana bandana".
```

```
       01 N         PIC 99.
       PROCEDURE DIVISION.
           MOVE 1 TO PTR.
           STRING "Dr. " DELIMITED BY SIZE
                  "Grace Hopper" DELIMITED BY SPACE
                  " of the Navy" DELIMITED BY SIZE
               INTO FULL WITH POINTER PTR
               ON OVERFLOW DISPLAY "overflow"
           END-STRING.
           DISPLAY FULL "|" PTR.
           MOVE SPACES TO PARTS.
           MOVE 1 TO PTR.
           UNSTRING LINE-IN DELIMITED BY "," OR ";"
               INTO PART (1) PART (2) PART (3) PART (4) PART (5)
               TALLYING IN NPARTS
           END-UNSTRING.
           DISPLAY NPARTS ":" PART (1) "|" PART (2) "|" PART (3)
               "|" PART (4) "|" PART (5).
           UNSTRING LINE-IN DELIMITED BY ALL "," OR ";"
               INTO PART (1) DELIMITER IN DLM COUNT IN CNT
           END-UNSTRING.
           DISPLAY PART (1) " delimiter " DLM " count " CNT.
           INSPECT TXT TALLYING N FOR ALL "an".
           DISPLAY N " times an".
           INSPECT TXT REPLACING ALL "an" BY "AN" AFTER INITIAL " ".
           DISPLAY TXT.
           INSPECT TXT CONVERTING "abn" TO "123".
           DISPLAY TXT.
           STOP RUN.

$ ./demo
Dr. Grace of the Navy         |22
05:10  |20  |30  |    |40
10   delimiter , count 02
04 times an
banana bANdANa
213131 2ANdAN1
```

Figure 43. STRING, UNSTRING and INSPECT

SET, INITIALIZE, ALLOCATE and FREE

`SET` has several forms: `SET index TO n`, `SET index UP BY n` and `DOWN BY n` for table indexes; `SET condition-name TO TRUE | FALSE` (Chapter 5); `SET pointer TO ADDRESS OF item`, `SET ADDRESS OF item TO pointer`, and `SET pointer TO NULL`; and `SET object TO NULL`.

`INITIALIZE item ...` gives every elementary part of the item a start value: zero for numbers and blanks for alphanumeric items, except FILLER items, which are left alone, and parts that REDEFINE another. `REPLACING category DATA BY value` sets all parts of a category (ALPHABETIC, ALPHANUMERIC, NUMERIC, NATIONAL, ALPHANUMERIC-EDITED, NUMERIC-EDITED) to another value, and `WITH FILLER`, `ALL TO VALUE` and `TO DEFAULT` extend the statement.

`ALLOCATE item [INITIALIZED] [RETURNING pointer]` gets storage from the heap for a BASED item, or, when `ALLOCATE n CHARACTERS` is written, for n bytes and returns the address. `FREE pointer` gives it back.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 REC.
          05 REC-NAME  PIC X(6) VALUE "OLD".
          05 FILLER    PIC X    VALUE "|".
          05 REC-QTY   PIC 99   VALUE 42.
       01 P-REC        POINTER.
       01 BUF          PIC X(10) BASED.
       01 PTR2         POINTER.
       PROCEDURE DIVISION.
           INITIALIZE REC.
           DISPLAY "[" REC "]".
           INITIALIZE REC REPLACING ALPHANUMERIC DATA BY "NEW"
               NUMERIC DATA BY 7.
           DISPLAY "[" REC "]".
           SET P-REC TO ADDRESS OF REC.
           IF P-REC NOT = NULL DISPLAY "pointer set" END-IF.
           ALLOCATE BUF RETURNING PTR2.
           MOVE "allocated" TO BUF.
           DISPLAY BUF.
           FREE PTR2.
           SET PTR2 TO NULL.
           IF PTR2 = NULL DISPLAY "freed" END-IF.
           STOP RUN.

$ ./demo
[      |00]
[NEW   |07]
pointer set
allocated
freed
```

Figure 44. SET, INITIALIZE and ALLOCATE

Chapter 7. Conditions and Expressions

Arithmetic expressions

An arithmetic expression combines numeric fields, numeric literals and functions with the operators `+`, `-`, `*`, `/` and `**` (exponentiation), and parentheses. Unary `+` and `-` are allowed. `**` binds more tightly than `*` and `/`, which bind more tightly than `+` and `-`; operators of the same rank go from left to right. Expressions are used by COMPUTE, in conditions, in subscripts, in reference modification, and as operands of the other arithmetic statements and of IF.

The evaluation is exact: each intermediate result keeps all its digits, up to 80, and a division is carried to 34 decimal places. The result is cut or rounded only when it is stored into a field. The famous sum 0.1 + 0.2 is therefore exactly 0.3, and an equality test of it succeeds.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 A        PIC 9V9 VALUE 0.1.
       01 B        PIC 9V9 VALUE 0.2.
       01 BIG      PIC 9(18) VALUE 999999999999999999.
       01 PROD     PIC 9(36).
       01 THIRD    PIC 9V9(30).
       01 R        PIC S9(3)V99.
       PROCEDURE DIVISION.
           IF A + B = 0.3 DISPLAY "0.1 + 0.2 = 0.3" END-IF.
           COMPUTE PROD = BIG * BIG.
           DISPLAY PROD.
           COMPUTE THIRD = 1 / 3.
           DISPLAY THIRD.
           COMPUTE R = 2 ** 3 ** 2 / 4 - -1.
           DISPLAY R.
           COMPUTE R = -7 / 2.
           DISPLAY R.
           COMPUTE R = 2 ** 0.5.
           DISPLAY R.
           STOP RUN.

$ ./demo
0.1 + 0.2 = 0.3
999999999999999998000000000000000001
0.33333333333333333333333333333333
+129.00
-003.50
+001.41
```

Figure 45. Exact decimal arithmetic

In the example `2 ** 3 ** 2` is taken as `2 ** (3 ** 2)` = 512: repeated powers group to the right, as in mathematics. 512 / 4 - -1 is 129.

Conditions

A **simple condition** is a relation, a class test, a sign test, a condition name, or an expression in parentheses. Conditions are joined by `AND` and `OR` and reversed by `NOT`; `AND` binds more tightly than `OR`.

Written as	Symbol	Meaning
`IS EQUAL TO`, `IS =`	`=`	Equal
`IS NOT EQUAL TO`	`NOT =`, `<>`	Not equal

Written as	Symbol	Meaning
`IS GREATER THAN`	`>`	Greater
`IS LESS THAN`	`<`	Less
`IS GREATER THAN OR EQUAL TO`	`>=`	Greater or equal
`IS LESS THAN OR EQUAL TO`	`<=`	Less or equal

Figure 46. Relation operators

Two numeric operands are compared by value, whatever their usage and number of decimal places. Two alphanumeric operands are compared character by character with the collating sequence of the machine; the shorter one is extended with blanks. A numeric field compared with an alphanumeric literal is compared as characters. The relation may be **abbreviated**: in `A > B AND C` the subject A and the operator `>` are repeated for C, and `A = 1 OR 2 OR 3` tests three values.

Test	True when
`item IS NUMERIC`	the item holds only digits (with a valid sign if it is signed).
`item IS ALPHABETIC`	the item holds only letters and blanks. `ALPHABETIC-LOWER` and `ALPHABETIC-UPPER` accept only lower case or only upper case letters (and blanks).
`item IS POSITIVE`	the numeric item or expression is greater than zero. `NEGATIVE` is less than zero and `ZERO` is zero.
`item IS NOT class`	the test is false.
`ptr = NULL`	a pointer is the null pointer.

Figure 47. Class and sign tests

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 A        PIC 99    VALUE 15.
       01 B        PIC 99V9  VALUE 15.0.
       01 TXT-1    PIC X(5)  VALUE "ABC".
       01 TXT-2    PIC X(3)  VALUE "ABC".
       01 DIGITS   PIC X(4)  VALUE "12A4".
       01 NUM-STR  PIC X(4)  VALUE "1234".
       01 DIFF     PIC S99   VALUE -5.
       PROCEDURE DIVISION.
           IF A = B DISPLAY "15 equals 15.0" END-IF.
           IF TXT-1 = TXT-2 DISPLAY "ABC equals ABC with blanks" END-IF.
           IF DIGITS IS NUMERIC DISPLAY "digits" ELSE
               DISPLAY "DIGITS is not numeric" END-IF.
           IF NUM-STR NUMERIC DISPLAY "NUM-STR is numeric" END-IF.
           IF TXT-2 IS ALPHABETIC-UPPER DISPLAY "upper case" END-IF.
           IF DIFF IS NEGATIVE DISPLAY "negative" END-IF.
           IF A > 10 AND A < 20 OR A = 99 DISPLAY "in range" END-IF.
           IF A = 1 OR 15 OR 30 DISPLAY "one of three" END-IF.
           IF A > 20 AND < 30 OR NOT = 15 DISPLAY "no" ELSE
               DISPLAY "abbreviated relations work" END-IF.
           IF NOT (A < 10) DISPLAY "not less than 10" END-IF.
           STOP RUN.

$ ./demo
15 equals 15.0
ABC equals ABC with blanks
DIGITS is not numeric
NUM-STR is numeric
upper case
negative
in range
one of three
abbreviated relations work
not less than 10
```

Figure 48. Conditions

Chapter 8. Tables

A table is an item with an OCCURS clause (Chapter 5). All elements have the same description; a table can contain tables, to three levels. An element is named with a **subscript** in parentheses, which counts from 1: `PRICE (3)`. A subscript is an integer literal, a numeric field, an index name, or an arithmetic expression with the form `name + 1` or `name - 1`. A table of groups is used as `ITEM-PRICE OF LINE (I)`; a table within a table as `CELL (ROW, COL)` or `CELL (ROW COL)`.

Subscripts and indexes

An **index** is declared by `INDEXED BY name` on the OCCURS clause. It needs no other description. An index is set with `SET`, and it can be used as a subscript of its table, in a comparison, and in PERFORM VARYING. It holds a position, and the program should not use it as a number: use `SET number TO index` to get the position in a numeric field.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 SALES.
          05 MONTH-SALES OCCURS 4 TIMES INDEXED BY MX.
             10 UNITS    PIC 9(3).
             10 PRICE    PIC 9(2)V99.
       01 GRID.
          05 ROW OCCURS 2 TIMES.
             10 CELL     PIC 9 OCCURS 3 TIMES.
       01 I         PIC 9.
       01 J         PIC 9.
       01 POS        PIC 9.
       PROCEDURE DIVISION.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > 4
               COMPUTE UNITS (I) = I * 10
               COMPUTE PRICE (I) = I + 0.5
           END-PERFORM.
           DISPLAY SALES.
           SET MX TO 3.
           DISPLAY UNITS (MX) " " PRICE (MX + 1).
           SET MX DOWN BY 1.
           SET POS TO MX.
           DISPLAY "index at " POS.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > 2
               PERFORM VARYING J FROM 1 BY 1 UNTIL J > 3
                   COMPUTE CELL (I, J) = I * 3 + J
               END-PERFORM
           END-PERFORM.
           DISPLAY GRID " " ROW (2).
           STOP RUN.

$ ./demo
010015002002500300350040045O
030 04.50
index at 2
456789 789
```

Figure 49. Subscripts and indexes

A subscript outside the table is not detected by the generated program, and the access goes to the storage beside the table. For the testing of a program the environment variable `COB_CHECK=1` makes the program test every

subscript and every reference modification at run time and stop with the message `subscript out of range` (Chapter 14). The translator option `-check` is accepted, but in this release it has no effect.

Tables of variable length

`OCCURS min TO max TIMES DEPENDING ON n` reserves room for max elements. The current length is the value of n, which the program has to keep in the range min to max. The length is used by SEARCH, by SORT of the table, and by the size of a variable-length record in a file.

SEARCH

`SEARCH table [VARYING index] [AT END statements] WHEN condition statements ... END-SEARCH` is a linear search. It begins at the element that the index of the table now points at, tests the WHEN conditions for each element, and stops at the first that is true; the statements of that WHEN are performed with the index pointing at the element. If the end of the table is reached, AT END is performed. The index has to be set before the search; the usual start is `SET index TO 1`.

`SEARCH ALL table AT END ... WHEN key = value ... END-SEARCH` searches a table that is sorted on the key of its ASCENDING KEY or DESCENDING KEY clause. In this release the search scans the table from its first element, so the table has to be in key order for the standard result and the same result is given in every other respect.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 CODES.
          05 FILLER PIC X(12) VALUE "AT ATLANTA  ".
          05 FILLER PIC X(12) VALUE "BO BOSTON   ".
          05 FILLER PIC X(12) VALUE "CH CHICAGO  ".
          05 FILLER PIC X(12) VALUE "DE DENVER   ".
       01 CITY-TABLE REDEFINES CODES.
          05 CITY OCCURS 4 TIMES
                ASCENDING KEY IS CITY-CODE INDEXED BY CX.
             10 CITY-CODE   PIC XX.
             10 FILLER      PIC X.
             10 CITY-NAME   PIC X(9).
       01 WANT         PIC XX.
       PROCEDURE DIVISION.
           MOVE "CH" TO WANT.
           SET CX TO 1.
           SEARCH CITY
               AT END DISPLAY WANT " not found"
               WHEN CITY-CODE (CX) = WANT
                   DISPLAY WANT " is " CITY-NAME (CX)
           END-SEARCH.
           MOVE "DE" TO WANT.
           SEARCH ALL CITY
               AT END DISPLAY WANT " not found"
               WHEN CITY-CODE (CX) = WANT
                   DISPLAY WANT " is " CITY-NAME (CX)
           END-SEARCH.
           MOVE "ZZ" TO WANT.
           SET CX TO 1.
           SEARCH CITY
               AT END DISPLAY WANT " not found"
               WHEN CITY-CODE (CX) = WANT DISPLAY "found"
           END-SEARCH.
           STOP RUN.

$ ./demo
CH is CHICAGO
DE is DENVER
ZZ not found
```

Figure 50. SEARCH and SEARCH ALL

Sorting a table

`SORT table ON ASCENDING | DESCENDING KEY key ...` puts the elements of a table in order. The keys are fields of the element. The sort is stable: elements with equal keys keep their order. If the table has a DEPENDING ON item, only the current elements are sorted. The KEY phrase is required; `SORT table` alone leaves the table as it is. The same statement sorts files (Chapter 11).

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 N          PIC 9 VALUE 5.
       01 TBL.
          05 ENTRY-ITEM OCCURS 1 TO 8 TIMES DEPENDING ON N
                ASCENDING KEY IS KEY-1.
             10 KEY-1     PIC 9.
             10 TAG       PIC X.
       01 I          PIC 9.
       PROCEDURE DIVISION.
           MOVE "3a1b3c2d1e" TO TBL.
           SORT ENTRY-ITEM ON ASCENDING KEY KEY-1.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > N
               DISPLAY KEY-1 (I) TAG (I) " " WITH NO ADVANCING
           END-PERFORM.
           DISPLAY SPACE.
           SORT ENTRY-ITEM ON DESCENDING KEY KEY-1.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > N
               DISPLAY KEY-1 (I) TAG (I) " " WITH NO ADVANCING
           END-PERFORM.
           DISPLAY SPACE.
           STOP RUN.

$ ./demo
1b 1e 2d 3a 3c
3a 3c 2d 1b 1e
```

Figure 51. Sorting a table

Chapter 9. Intrinsic Functions

An intrinsic function is used where an operand is allowed: `FUNCTION name (argument ...)`. The arguments are separated by blanks or commas, and the parentheses are left out when there is no argument (`FUNCTION PI`). A function returns a number or an alphanumeric string, and can appear in MOVE, DISPLAY, COMPUTE, IF, as an argument of another function, and with reference modification (`FUNCTION TRIM (X) (1:3)`). A program with the entry `REPOSITORY. FUNCTION ALL INTRINSIC.` may write the function names without the word FUNCTION. The table lists every function that the translator knows.

Group	Functions
Strings	CONCATENATE, LENGTH, BYTE-LENGTH, LOWER-CASE, UPPER-CASE, REVERSE, TRIM, TRIM-LEFT, TRIM-RIGHT, SUBSTITUTE, SUBSTITUTE-CASE, STORED-CHAR-LENGTH, CHAR, ORD, HEX-OF, BASECONVERT
Numbers	ABS, SIGN, INTEGER, INTEGER-PART, FRACTION-PART, MOD, REM, FACTORIAL, NUMVAL, NUMVAL-C, NUMVAL-F, TEST-NUMVAL, TEST-NUMVAL-C, TEST-NUMVAL-F, RANDOM
Statistics	MAX, MIN, MEAN, MEDIAN, MIDRANGE, RANGE, SUM, VARIANCE, STANDARD-DEVIATION, ORD-MAX, ORD-MIN
Mathematics	SQRT, EXP, EXP10, LOG, LOG10, SIN, COS, TAN, ASIN, ACOS, ATAN, PI, E, HIGHEST-ALGEBRAIC, LOWEST-ALGEBRAIC
Finance	ANNUITY, PRESENT-VALUE
Dates and times	CURRENT-DATE, WHEN-COMPILED, DATE-OF-INTEGER, DAY-OF-INTEGER, INTEGER-OF-DATE, INTEGER-OF-DAY, DATE-TO-YYYYMMDD, DAY-TO-YYYYDDD, YEAR-TO-YYYY, TEST-DATE-YYYYMMDD, TEST-DAY-YYYYDDD, FORMATTED-CURRENT-DATE, FORMATTED-DATE, FORMATTED-TIME, FORMATTED-DATETIME, INTEGER-OF-FORMATTED-DATE, SECONDS-FROM-FORMATTED-TIME, SECONDS-PAST-MIDNIGHT, TEST-FORMATTED-DATETIME, COMBINED-DATETIME
Bits and booleans	BOOLEAN-OF-INTEGER, INTEGER-OF-BOOLEAN, BIT-OF, BIT-TO-CHAR
National characters	DISPLAY-OF, NATIONAL-OF, CHAR-NATIONAL, ULENGTH, UPOS, USUBSTR
System	NUMBER-OF-CALL-PARAMETERS, UUID4, LOCALE-DATE, LOCALE-TIME, MODULE-NAME, MODULE-ID, EXCEPTION-FILE, EXCEPTION-LOCATION, EXCEPTION-STATEMENT, EXCEPTION-STATUS

Figure 52. Intrinsic functions by group

String functions

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 NAME     PIC X(14) VALUE "  Ada Lovelace".
       01 PART     PIC X(5).
       PROCEDURE DIVISION.
           DISPLAY "[" FUNCTION TRIM (NAME) "]".
           DISPLAY "[" FUNCTION TRIM-LEFT (NAME) "]".
           DISPLAY FUNCTION UPPER-CASE (NAME).
           DISPLAY FUNCTION LOWER-CASE (NAME).
           DISPLAY FUNCTION REVERSE (FUNCTION TRIM (NAME)).
           DISPLAY FUNCTION LENGTH (NAME) " "
               FUNCTION LENGTH (FUNCTION TRIM (NAME)).
           DISPLAY FUNCTION CONCATENATE ("one", "-", "two").
           DISPLAY FUNCTION SUBSTITUTE ("banana", "an", "AN").
           DISPLAY FUNCTION ORD ("A") " " FUNCTION CHAR (66).
           MOVE FUNCTION TRIM (NAME) (1:3) TO PART.
           DISPLAY PART "|".
           STOP RUN.

$ ./demo
[Ada Lovelace]
[Ada Lovelace]
  ADA LOVELACE
  ada lovelace
ecalevoL adA
14 12
one-two
bANANa
66 A
Ada  |
```

Figure 53. String functions

`ORD` returns the position of a character in the collating sequence counted from 1, so `ORD ("A")` is 66 on an ASCII machine, and `CHAR (66)` gives back the letter A. The lengths of a field in `LENGTH` and `BYTE-LENGTH` are the same for single-byte data. `TRIM` removes blanks at both ends, `TRIM-LEFT` at the start and `TRIM-RIGHT` at the end; the standard phrase `TRIM (x LEADING)` is not accepted.

Numeric functions

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 X        PIC S9(3)V99 VALUE -7.25.
       PROCEDURE DIVISION.
           DISPLAY FUNCTION ABS (X) " " FUNCTION SIGN (X).
           DISPLAY FUNCTION INTEGER (X) " " FUNCTION INTEGER-PART (X)
               " " FUNCTION FRACTION-PART (X).
           DISPLAY FUNCTION MOD (-11, 5) " " FUNCTION REM (-11, 5).
           DISPLAY FUNCTION MAX (3, 9, 4) " " FUNCTION MIN (3, 9, 4).
           DISPLAY FUNCTION MEAN (1, 2, 3, 4) " "
               FUNCTION MEDIAN (5, 1, 3) " " FUNCTION RANGE (5, 1, 3).
           DISPLAY FUNCTION SQRT (2).
           DISPLAY FUNCTION PI.
           DISPLAY FUNCTION FACTORIAL (20).
           DISPLAY FUNCTION NUMVAL (" 12.50- ") " "
               FUNCTION NUMVAL-C ("$1,234.50").
           COMPUTE X ROUNDED = FUNCTION SQRT (50).
           DISPLAY X.
           STOP RUN.

$ ./demo
7.25 -1
-8 -7 -0.25
4 -1
9 3
2.5 3 4
1.41421356237309514547462185873 9
3.14159265358979323846264338327950 29
2432902008176640000
-12.5 1234.5
+007.07
```

Figure 54. Numeric functions

The mathematical functions work through the C library: SQRT, EXP, LOG, SIN and the others are computed in the `double` type, so their results are reliable to about 15 digits, not 34; the digits that DISPLAY shows after the sixteenth are noise from the binary conversion, as in `SQRT (2)` above. Everything else is done with the exact decimal engine. `RANDOM` returns a number between 0 and 1; with an argument it sets the seed.

Date functions

Dates are held in the forms YYYYMMDD (calendar date), YYYYDDD (day of the year) and an integer that counts days from 1 January 1601 (day 1). `INTEGER-OF-DATE`, `DATE-OF-INTEGER`, `INTEGER-OF-DAY` and `DAY-OF-INTEGER` convert between them, so that date arithmetic is integer arithmetic. `CURRENT-DATE` returns 21 characters, YYYYMMDDhhmmsshh followed by the offset from UTC.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 D1      PIC 9(8) VALUE 20261012.
       01 N1      PIC 9(7).
       01 NOW      PIC X(21).
       PROCEDURE DIVISION.
           COMPUTE N1 = FUNCTION INTEGER-OF-DATE (D1).
           DISPLAY D1 " is day " N1.
           DISPLAY "in 90 days: "
               FUNCTION DATE-OF-INTEGER (N1 + 90).
           DISPLAY "day of year: " FUNCTION DAY-OF-INTEGER (N1).
           DISPLAY "weekday: "
               FUNCTION MOD (N1, 7) " (0 = Sunday)".
           DISPLAY "2-digit year 26: " FUNCTION YEAR-TO-YYYY (26).
           DISPLAY "valid? " FUNCTION TEST-DATE-YYYYMMDD (20260231)
               " " FUNCTION TEST-DATE-YYYYMMDD (20260228).
           MOVE FUNCTION CURRENT-DATE TO NOW.
           DISPLAY "century " NOW (1:2).
           STOP RUN.

$ ./demo
20261012 is day 0155513
in 90 days: 20270110
day of year: 2026285
weekday: 1 (0 = Sunday)
2-digit year 26: 2026
valid? 3 0
century 20
```

Figure 55. Date functions

`TEST-DATE-YYYYMMDD` returns 0 for a valid date, and otherwise the position of the first wrong part: 1 for the year, 2 for the month, 3 for the day.

Functions with limits

`EXCEPTION-FILE`, `EXCEPTION-LOCATION`, `EXCEPTION-STATEMENT` and `EXCEPTION-STATUS` return blanks, because the program keeps no exception record. `LOCALE-DATE` and `LOCALE-TIME` use fixed formats, and `MODULE-NAME` and `MODULE-ID` return the string COBOL. A name that is not in the table gives the translator error `unknown function`, or the run-time error `intrinsic function not implemented`.

Chapter 10. Files

A COBOL program names its files in the FILE-CONTROL paragraph of the environment division, describes their records in the FILE SECTION of the data division, and reads and writes them with the statements OPEN, READ, WRITE, REWRITE, DELETE, START and CLOSE. cobol has four kinds of files, which differ in the way records are found.

Organization	Access	On disk
`SEQUENTIAL`	Records are read in the order in which they were written.	Records follow each other without any separator. A file with variable-length records has a 4-byte length in front of each record.
`LINE SEQUENTIAL`	As sequential. The records are text lines.	One line per record, ended by a newline character. Trailing blanks are not written.
`RELATIVE`	Sequential, or random by record number (`RELATIVE KEY`).	A slot per record number: one flag byte (1 for a present record, 0 for an empty slot) followed by the record.
`INDEXED`	Sequential in key order, or random by key (`RECORD KEY`), with up to eight alternate keys.	A flat file of the records in the order of the primary key.

Figure 56. File organizations

The SELECT entry

```
SELECT [OPTIONAL] file-name
    ASSIGN TO external-name
    [ORGANIZATION IS SEQUENTIAL | LINE SEQUENTIAL | RELATIVE | INDEXED]
    [ACCESS MODE IS SEQUENTIAL | RANDOM | DYNAMIC]
    [RECORD KEY IS key]
    [ALTERNATE RECORD KEY IS key [WITH DUPLICATES]] ...
    [RELATIVE KEY IS data-name]
    [FILE STATUS IS data-name].
```

Figure 57. The SELECT entry

`ASSIGN TO` takes a literal (`"customers.dat"`) or the name of a data item that holds the file name when the file is opened. The name `ASSIGN TO DISK "x"` is accepted as well. A bare word that is not a data item, as in the JCL style `ASSIGN TO OUTDD`, is rejected by the translator; write a literal in quotes. If a program names no file at all in ASSIGN, the name of the SELECT is used. The default is organization sequential and access sequential.

A file name that contains no slash can be mapped to another path at run time by an environment variable: `DD_name`, `dd_name` or `name` (the order of the search). Programs thus need not change when the data live in another directory. The clauses `LOCK MODE`, `RESERVE`, `SHARING` and `PADDING` are accepted and have no effect. The file of a sort is described by `SD` (Chapter 11).

How the files are stored

The next program writes the record `AB1234` to a sequential, a line sequential and a relative file (as record 2), and two records to an indexed file, and then dumps the files with `od`. The indexed file shows the records in key order; the line sequential file shows the newline and that the short second record was written without trailing blanks.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT FSEQ ASSIGN TO "seq.dat" ORGANIZATION SEQUENTIAL.
```

```
           SELECT FLINE ASSIGN TO "line.dat"
               ORGANIZATION LINE SEQUENTIAL.
           SELECT FREL ASSIGN TO "rel.dat" ORGANIZATION RELATIVE
               ACCESS MODE RANDOM RELATIVE KEY RK.
           SELECT FIDX ASSIGN TO "idx.dat" ORGANIZATION INDEXED
               ACCESS MODE DYNAMIC RECORD KEY IKEY.
       DATA DIVISION.
       FILE SECTION.
       FD FSEQ.
       01 SEQ-REC     PIC X(6).
       FD FLINE.
       01 LINE-REC    PIC X(6).
       FD FREL.
       01 REL-REC     PIC X(6).
       FD FIDX.
       01 IDX-REC.
          05 IKEY     PIC XX.
          05 IVAL     PIC X(4).
       WORKING-STORAGE SECTION.
       01 RK PIC 9.
       PROCEDURE DIVISION.
           OPEN OUTPUT FSEQ FLINE FREL FIDX.
           MOVE "AB1234" TO SEQ-REC LINE-REC REL-REC.
           WRITE SEQ-REC. WRITE LINE-REC.
           MOVE "CD56" TO LINE-REC. WRITE LINE-REC.
           MOVE 2 TO RK. WRITE REL-REC.
           MOVE "ZZ9999" TO IDX-REC. WRITE IDX-REC.
           MOVE "AA1111" TO IDX-REC. WRITE IDX-REC.
           CLOSE FSEQ FLINE FREL FIDX.
           STOP RUN.

$ ./demo

$ od -An -c seq.dat
   A   B   1   2   3   4
$ od -An -c line.dat
   A   B   1   2   3   4  \n   C   D   5   6  \n
$ od -An -c rel.dat
  \0  \0  \0  \0  \0  \0  \0 001   A   B   1   2   3   4
$ od -An -c idx.dat
   A   A   1   1   1   1   Z   Z   9   9   9   9
```

Figure 58. The four file formats

An indexed file is read completely into memory when it is opened and written back when it is closed. Alternate keys are built when the file is opened, so they take no room on disk. The consequence is that a program that is stopped before CLOSE loses the records it wrote since OPEN; always close an indexed file.

The statements

Statement	Action
`OPEN INPUT | OUTPUT | I-O | EXTEND file ...`	Opens the files. OUTPUT creates an empty file. EXTEND adds to the end of a sequential file. I-O needs an existing file (status 35 otherwise).
`CLOSE file ...`	Closes the files. An indexed file is written back to disk here.
`READ file [NEXT] [INTO item] [AT END ...] [INVALID KEY ...]`	Reads the next record, or, in random or dynamic access of a relative or indexed file, the record whose key is in the key item. `AT END` is for sequential reading, `INVALID KEY` for reading by key. `INTO` moves the record to the item.
`WRITE record [FROM item] [BEFORE | AFTER ADVANCING n LINES] [INVALID KEY ...]`	Adds a record. `ADVANCING` is accepted for line sequential files. A duplicate key gives status 22.

Statement	Action
`REWRITE record [FROM item]`	Replaces the record read last. In an indexed file the primary key must not have been changed.
`DELETE file`	Removes the record read last (relative and indexed files).
`START file KEY IS = | > | >= | < | <= key [INVALID KEY ...]`	Sets the position in an indexed or relative file. The next sequential READ gets the first record that satisfies the condition. Status 23 if there is none.

Figure 59. File statements

A WRITE and a REWRITE name the **record**, not the file. A statement can have `AT END` or `INVALID KEY` phrases with `NOT AT END` and `NOT INVALID KEY`, and the matching terminator, `END-READ`, `END-WRITE`, `END-REWRITE`, `END-DELETE`, `END-START`.

The next program, `examples/report.cob`, reads a line sequential file of customers and writes a report. Its statement `WRITE REPORT-LINE FROM DETAIL` moves the group DETAIL to the record and writes it.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. REPORT1.
      * Reads a line sequential customer file and prints a report
      * with a page heading, one line per customer and a total.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT CUST-FILE ASSIGN TO "customers.dat"
               ORGANIZATION IS LINE SEQUENTIAL
               FILE STATUS IS CUST-STATUS.
           SELECT REPORT-FILE ASSIGN TO "report.txt"
               ORGANIZATION IS LINE SEQUENTIAL.
       DATA DIVISION.
       FILE SECTION.
       FD CUST-FILE.
       01 CUST-REC.
          05 CUST-NO      PIC 9(4).
          05 CUST-NAME    PIC X(18).
          05 CUST-BALANCE PIC 9(5)V99.
       FD REPORT-FILE.
       01 REPORT-LINE     PIC X(40).
       WORKING-STORAGE SECTION.
       01 CUST-STATUS     PIC XX.
       01 END-FLAG        PIC X       VALUE "N".
          88 AT-END                   VALUE "Y".
       01 GRAND-TOTAL     PIC 9(7)V99 VALUE ZERO.
       01 COUNT-READ      PIC 999     VALUE ZERO.
       01 HEAD-1          PIC X(40)
                          VALUE "CUSTOMER BALANCES".
       01 HEAD-2          PIC X(40)
                          VALUE "NO.   NAME                BALANCE".
       01 DETAIL.
          05 DET-NO       PIC 9(4).
          05 FILLER       PIC XX      VALUE SPACES.
          05 DET-NAME     PIC X(18).
          05 DET-BALANCE  PIC ZZ,ZZ9.99.
          05 FILLER       PIC X(3)    VALUE SPACES.
          05 DET-FLAG     PIC X(5).
       01 TOTAL-LINE.
          05 FILLER       PIC X(21)   VALUE "TOTAL".
          05 TOT-BALANCE  PIC Z,ZZZ,ZZ9.99.
       PROCEDURE DIVISION.
       MAIN-PARA.
           OPEN INPUT CUST-FILE OUTPUT REPORT-FILE.
```

```
           IF CUST-STATUS NOT = "00"
               DISPLAY "cannot open customers.dat, status " CUST-STATUS
               STOP RUN
           END-IF.
           WRITE REPORT-LINE FROM HEAD-1.
           WRITE REPORT-LINE FROM HEAD-2.
           PERFORM READ-CUSTOMER.
           PERFORM UNTIL AT-END
               PERFORM PRINT-CUSTOMER
               PERFORM READ-CUSTOMER
           END-PERFORM.
           MOVE GRAND-TOTAL TO TOT-BALANCE.
           WRITE REPORT-LINE FROM TOTAL-LINE.
           CLOSE CUST-FILE REPORT-FILE.
           DISPLAY COUNT-READ " customers listed".
           STOP RUN.
       READ-CUSTOMER.
           READ CUST-FILE
               AT END SET AT-END TO TRUE
           END-READ.
       PRINT-CUSTOMER.
           ADD 1 TO COUNT-READ.
           ADD CUST-BALANCE TO GRAND-TOTAL.
           MOVE CUST-NO TO DET-NO.
           MOVE CUST-NAME TO DET-NAME.
           MOVE CUST-BALANCE TO DET-BALANCE.
           IF CUST-BALANCE = ZERO
               MOVE "PAID" TO DET-FLAG
           ELSE
               MOVE SPACES TO DET-FLAG
           END-IF.
           WRITE REPORT-LINE FROM DETAIL.

$ ./report
005 customers listed
$ cat report.txt
CUSTOMER BALANCES
NO.   NAME                BALANCE
1001  ADAMS, JOHN        1,250.00
1002  BAKER, MARY          875.50
1003  CLARK, PETER           0.00   PAID
1004  DAVIS, ALICE       2,310.75
1005  EVANS, ROBERT        499.00
TOTAL                    4,935.25
```

Figure 60. A report from a sequential file

The second example, `examples/custmaint.cob`, maintains an indexed file with an alternate key that allows duplicates. It adds records, rejects a duplicate key, changes and deletes records, positions with START and lists the file in the order of both keys.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. CUSTMAINT.
      * Maintenance of an indexed file of customers with an alternate
      * key: add, change, delete, look up by key, list by name.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT CUST ASSIGN TO "cust.idx"
               ORGANIZATION IS INDEXED
               ACCESS MODE IS DYNAMIC
               RECORD KEY IS C-ID
```

```
               ALTERNATE RECORD KEY IS C-NAME WITH DUPLICATES
               FILE STATUS IS ST.
       DATA DIVISION.
       FILE SECTION.
       FD CUST.
       01 C-REC.
          05 C-ID        PIC 9(4).
          05 C-NAME      PIC X(12).
          05 C-CITY      PIC X(10).
       WORKING-STORAGE SECTION.
       01 ST             PIC XX.
       01 MORE           PIC X.
          88 NO-MORE     VALUE "N".
       PROCEDURE DIVISION.
       MAIN-PARA.
           OPEN OUTPUT CUST.
           MOVE 1003 TO C-ID. MOVE "SMITH" TO C-NAME.
           MOVE "BERN" TO C-CITY. PERFORM ADD-REC.
           MOVE 1001 TO C-ID. MOVE "JONES" TO C-NAME.
           MOVE "ZURICH" TO C-CITY. PERFORM ADD-REC.
           MOVE 1002 TO C-ID. MOVE "SMITH" TO C-NAME.
           MOVE "BASEL" TO C-CITY. PERFORM ADD-REC.
           MOVE 1004 TO C-ID. MOVE "TEMP" TO C-NAME.
           MOVE "LUGANO" TO C-CITY. PERFORM ADD-REC.
           MOVE 1002 TO C-ID. MOVE "DUPLICATE" TO C-NAME.
           PERFORM ADD-REC.
           CLOSE CUST.
           OPEN I-O CUST.
           MOVE 1001 TO C-ID.
           READ CUST
               INVALID KEY DISPLAY "no customer 1001"
               NOT INVALID KEY
                   MOVE "GENEVA" TO C-CITY
                   REWRITE C-REC
                   DISPLAY "changed 1001"
           END-READ.
           MOVE 1004 TO C-ID.
           DELETE CUST
               INVALID KEY DISPLAY "cannot delete 1004"
           END-DELETE.
           MOVE 9999 TO C-ID.
           READ CUST INVALID KEY DISPLAY "no customer 9999" END-READ.
           DISPLAY "-- by key".
           MOVE LOW-VALUES TO C-ID.
           START CUST KEY IS NOT LESS THAN C-ID.
           PERFORM LIST-NEXT.
           DISPLAY "-- by name".
           MOVE "A" TO C-NAME.
           START CUST KEY IS NOT LESS THAN C-NAME.
           PERFORM LIST-NEXT.
           CLOSE CUST.
           STOP RUN.
       ADD-REC.
           WRITE C-REC
               INVALID KEY DISPLAY "duplicate key " C-ID " status " ST
           END-WRITE.
       LIST-NEXT.
           MOVE "Y" TO MORE.
           PERFORM UNTIL NO-MORE
               READ CUST NEXT
                   AT END MOVE "N" TO MORE
                   NOT AT END DISPLAY C-ID " " C-NAME " " C-CITY
```

```
               END-READ
           END-PERFORM.

$ ./custmaint
duplicate key 1002 status 22
changed 1001
no customer 9999
-- by key
1001 JONES        GENEVA
1002 SMITH        BASEL
1003 SMITH        BERN
-- by name
1001 JONES        GENEVA
1002 SMITH        BASEL
1003 SMITH        BERN
```

Figure 61. An indexed file with an alternate key

The file status

If a SELECT entry has a `FILE STATUS` item, a two-character field, every file statement stores a status in it, and an error does not stop the program. The first character tells the class: 0 for success, 1 for the end of the file, 2 for an invalid key, 3 and 4 for permanent errors, and the program decides what to do. The statuses that the run-time system gives are in the next table.

Status	Meaning
00	The statement succeeded.
02	A record was written whose alternate key is a duplicate of one already present (allowed with duplicates).
04	A record was shorter than the record description; the rest is not filled.
05	OPEN of an OPTIONAL file that does not exist; the file reads as empty.
10	The end of the file: READ NEXT found no further record.
21	The keys of an indexed file written in sequential access are not in ascending order.
22	A record with this primary key exists already (WRITE or REWRITE).
23	No record has this key (READ, START, DELETE, REWRITE).
24	A relative key is zero or too large.
30	A permanent error.
34	A record is longer than the maximum.
35	OPEN INPUT or I-O of a file that does not exist.
37	The file cannot be opened in that mode.
41	OPEN of a file that is open.
42	CLOSE, READ or WRITE of a file that is not open.
43	REWRITE or DELETE of a sequential file without a READ before it.
47	READ of a file that is not open for input.
48	WRITE of a file that is not open for output.
49	REWRITE or DELETE of a file that is not open I-O.

Figure 62. File status codes

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
```

```
       FILE-CONTROL.
           SELECT MISSING ASSIGN TO "nofile.dat"
               ORGANIZATION SEQUENTIAL FILE STATUS IS ST.
           SELECT OPTIONAL OPT ASSIGN TO "nofile.dat"
               ORGANIZATION SEQUENTIAL FILE STATUS IS ST.
           SELECT IDXF ASSIGN TO "k.dat" ORGANIZATION INDEXED
               ACCESS MODE DYNAMIC RECORD KEY IKEY
               FILE STATUS IS ST.
       DATA DIVISION.
       FILE SECTION.
       FD MISSING.
       01 M-REC       PIC X(4).
       FD OPT.
       01 O-REC       PIC X(4).
       FD IDXF.
       01 IDX-REC.
          05 IKEY     PIC XX.
          05 IVAL     PIC X(4).
       WORKING-STORAGE SECTION.
       01 ST PIC XX.
       PROCEDURE DIVISION.
           OPEN INPUT MISSING.        DISPLAY "open missing     " ST.
           OPEN INPUT OPT.            DISPLAY "open optional    " ST.
           READ OPT.                  DISPLAY "read optional    " ST.
           CLOSE OPT.
           OPEN OUTPUT IDXF.          DISPLAY "open output      " ST.
           OPEN OUTPUT IDXF.          DISPLAY "open again       " ST.
           MOVE "K1ONE " TO IDX-REC.  WRITE IDX-REC.
                                      DISPLAY "write            " ST.
           WRITE IDX-REC.             DISPLAY "write duplicate  " ST.
           READ IDXF.                 DISPLAY "read, output     " ST.
           CLOSE IDXF.
           WRITE IDX-REC.             DISPLAY "write, closed    " ST.
           OPEN I-O IDXF.             DISPLAY "open i-o         " ST.
           MOVE "K9" TO IKEY.
           READ IDXF.                 DISPLAY "read missing key " ST.
           MOVE "K1" TO IKEY.
           READ IDXF.                 DISPLAY "read key K1      " ST.
           READ IDXF NEXT.            DISPLAY "read next at end " ST.
           CLOSE IDXF.
           OPEN INPUT IDXF.
           WRITE IDX-REC.             DISPLAY "write, input     " ST.
           CLOSE IDXF.
           STOP RUN.

$ ./demo
open missing     35
open optional    05
read optional    10
open output      00
open again       41
write            00
write duplicate  22
read, output     47
write, closed    42
open i-o         00
read missing key 23
read key K1      00
read next at end 10
write, input     48
```

Figure 63. File status codes

If a file has no FILE STATUS item and a statement fails with a status of 10 or higher that the program does not handle with AT END or INVALID KEY, the run-time system stops the program. The message names the file and the status, and the exit status is 1:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT INF ASSIGN TO "nofile.dat".
       DATA DIVISION.
       FILE SECTION.
       FD INF.
       01 IN-REC PIC X(4).
       PROCEDURE DIVISION.
           OPEN INPUT INF.
           DISPLAY "not reached".
           STOP RUN.

$ ./demo; echo "exit status $?"
cobol: runtime error: file nofile.dat (status = 35)
exit status 1
```

Figure 64. An unhandled file error

Declaratives

A program can have a procedure that runs whenever an input-output statement fails. Such procedures are written in the DECLARATIVES part at the start of the procedure division, in sections that begin with a USE statement: `USE AFTER STANDARD ERROR PROCEDURE ON file ... | INPUT | OUTPUT | I-O | EXTEND`. The word EXCEPTION may be written for ERROR. After the procedure has run, the program continues with the statement after the one that failed. The sections that are not declaratives must be sections as well.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT INF ASSIGN TO "nofile.dat" FILE STATUS IS ST.
       DATA DIVISION.
       FILE SECTION.
       FD INF.
       01 IN-REC PIC X(4).
       WORKING-STORAGE SECTION.
       01 ST PIC XX.
       PROCEDURE DIVISION.
       DECLARATIVES.
       INPUT-ERRORS SECTION.
           USE AFTER STANDARD EXCEPTION PROCEDURE ON INF.
       REPORT-IT.
           DISPLAY "declarative: status " ST.
       END DECLARATIVES.
       MAIN SECTION.
       MAIN-PARA.
           OPEN INPUT INF.
           DISPLAY "after OPEN".
           STOP RUN.

$ ./demo
declarative: status 35
after OPEN
```

Figure 65. A USE declarative

Variable-length records and the record area

When an FD has several 01 entries, they describe the same storage, and the longest one gives the size of the record area. A sequential file with variable-length records stores a length before each record, as the table above shows. A line sequential file always has variable lengths. `READ file INTO item` moves the record to another item after the read; `WRITE record FROM item` moves it before the write.

Chapter 11. Sorting and Merging

SORT puts the records of a file, or the records that the program supplies, in order. The file that the sort works on is a **sort file**, described by an SD entry in the FILE SECTION and named in a SELECT entry; its name in ASSIGN is only a label. The statement is:

```
SORT sort-file
    ON ASCENDING | DESCENDING KEY key ...  [ON ...]
    INPUT PROCEDURE IS section | USING file ...
    OUTPUT PROCEDURE IS section | GIVING file ...
```

Figure 66. The SORT statement

There are up to sixteen keys; the first named is the major key. `USING file` reads the records of the named files (opened and closed by the sort), and `INPUT PROCEDURE` is a section that supplies the records with `RELEASE record [FROM item]`. `GIVING file` writes the sorted records to files, and `OUTPUT PROCEDURE` is a section that takes them one by one with `RETURN sort-file [INTO item] AT END ... END-RETURN`. The two procedures are sections (or paragraphs); control must not leave them with a GO TO to the outside. All the records are held in memory, and the sort is stable: records with equal keys come out in the order in which they went in.

The next example releases records from a table, sorts them by region (ascending) and amount (descending) and prints a total at every change of region, a **control break**.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. SORTDEMO.
      * SORT with an input procedure that releases records built in
      * storage and an output procedure that prints a total for each
      * region (a control break).
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT SORT-FILE ASSIGN TO "sortwork".
       DATA DIVISION.
       FILE SECTION.
       SD SORT-FILE.
       01 SORT-REC.
          05 S-REGION    PIC X(5).
          05 S-AMOUNT    PIC 9(5)V99.
       WORKING-STORAGE SECTION.
       01 SALES-DATA.
          05 FILLER PIC X(12) VALUE "NORTH0012050".
          05 FILLER PIC X(12) VALUE "SOUTH0004000".
          05 FILLER PIC X(12) VALUE "NORTH0030000".
          05 FILLER PIC X(12) VALUE "EAST 0007575".
          05 FILLER PIC X(12) VALUE "SOUTH0010025".
          05 FILLER PIC X(12) VALUE "NORTH0001500".
       01 SALES-TABLE REDEFINES SALES-DATA.
          05 SALE OCCURS 6 TIMES.
             10 SALE-REGION PIC X(5).
             10 SALE-AMOUNT PIC 9(5)V99.
       01 I               PIC 9.
       01 PREV-REGION     PIC X(5) VALUE SPACES.
       01 REGION-TOTAL    PIC 9(6)V99 VALUE ZERO.
       01 SHOW-AMOUNT     PIC ZZ,ZZ9.99.
       01 EOF-FLAG        PIC X VALUE "N".
          88 NO-MORE      VALUE "Y".
       PROCEDURE DIVISION.
       MAIN-PARA.
           SORT SORT-FILE
               ON ASCENDING KEY S-REGION
```

```
               ON DESCENDING KEY S-AMOUNT
               INPUT PROCEDURE IS LOAD-SALES
               OUTPUT PROCEDURE IS PRINT-SALES.
           STOP RUN.
       LOAD-SALES SECTION.
           PERFORM VARYING I FROM 1 BY 1 UNTIL I > 6
               MOVE SALE-REGION (I) TO S-REGION
               MOVE SALE-AMOUNT (I) TO S-AMOUNT
               RELEASE SORT-REC
           END-PERFORM.
       PRINT-SALES SECTION.
           PERFORM UNTIL NO-MORE
               RETURN SORT-FILE
                   AT END SET NO-MORE TO TRUE
               END-RETURN
               IF NOT NO-MORE
                   IF S-REGION NOT = PREV-REGION
                       PERFORM REGION-BREAK
                       MOVE S-REGION TO PREV-REGION
                   END-IF
                   MOVE S-AMOUNT TO SHOW-AMOUNT
                   DISPLAY "  " S-REGION SHOW-AMOUNT
                   ADD S-AMOUNT TO REGION-TOTAL
               END-IF
           END-PERFORM.
           PERFORM REGION-BREAK.
       REGION-BREAK SECTION.
           IF PREV-REGION NOT = SPACES
               MOVE REGION-TOTAL TO SHOW-AMOUNT
               DISPLAY "  total" SHOW-AMOUNT
           END-IF.
           MOVE ZERO TO REGION-TOTAL.

$ ./sort
  EAST     75.75
  total    75.75
  NORTH   300.00
  NORTH   120.50
  NORTH    15.00
  total   435.50
  SOUTH   100.25
  SOUTH    40.00
  total   140.25
```

Figure 67. SORT with input and output procedures

`MERGE` has the same syntax as SORT with USING; it merges the files named in USING into the file named in GIVING or the output procedure. In this release the merge reads all the input files one after the other and sorts the result, which gives the right answer when the inputs are in order or not. The next example sorts a file with USING and GIVING, and merges two files. The file `in1.txt` holds the lines `3three`, `1one` and `2two`, and `in2.txt` holds `5five` and `4four`.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT IN-1 ASSIGN TO "in1.txt"
               ORGANIZATION LINE SEQUENTIAL.
           SELECT IN-2 ASSIGN TO "in2.txt"
               ORGANIZATION LINE SEQUENTIAL.
           SELECT OUT-F ASSIGN TO "out.txt"
```

```
               ORGANIZATION LINE SEQUENTIAL.
           SELECT WORK ASSIGN TO "work".
       DATA DIVISION.
       FILE SECTION.
       FD IN-1.
       01 IN-1-REC    PIC X(6).
       FD IN-2.
       01 IN-2-REC    PIC X(6).
       FD OUT-F.
       01 OUT-REC     PIC X(6).
       SD WORK.
       01 WORK-REC.
          05 W-KEY    PIC 9.
          05 W-TXT    PIC X(5).
       PROCEDURE DIVISION.
           SORT WORK ON ASCENDING KEY W-KEY
               USING IN-1 GIVING OUT-F.
           DISPLAY "sorted:".
           CALL "SYSTEM" USING "cat out.txt".
           MERGE WORK ON DESCENDING KEY W-KEY
               USING IN-1 IN-2 GIVING OUT-F.
           DISPLAY "merged:".
           CALL "SYSTEM" USING "cat out.txt".
           STOP RUN.

$ ./demo
sorted:
1one
2two
3three
merged:
5five
4four
3three
2two
1one
```

Figure 68. SORT USING GIVING and MERGE

The special register `SORT-RETURN` exists for compatibility; the sort gives no return codes. SORT of a table is described in Chapter 8.

Chapter 12. Subprograms, User Functions and C

CALL

`CALL name USING arg ... [RETURNING item] [ON EXCEPTION ... NOT ON EXCEPTION ...] END-CALL` calls another program. The name is a literal or a data item that holds the name. The called program has the LINKAGE SECTION for its arguments, a `PROCEDURE DIVISION USING parameter ...` header, and ends with `GOBACK` or `EXIT PROGRAM`. The first program of a source, or the one built last, is the main program; a program that is called is a subprogram.

Mode	Effect
`BY REFERENCE`	The default. The subprogram works on the storage of the caller, so changes are seen by the caller. A literal can be passed this way; the subprogram should not change it.
`BY CONTENT`	A copy of the argument is passed. The caller's item does not change.
`BY VALUE`	For calling C functions: the number is passed as a C `int`. The item must be a number.

Figure 69. Ways to pass an argument

The mode applies to all following arguments until another mode is named. `RETURNING item` receives the result of a function or subprogram. `RETURN-CODE` is a special register that the called program can set; it is shared with the caller and is the exit status of the main program. A called COBOL program keeps its WORKING-STORAGE between calls, so a counter in it goes on counting. Programs that are declared `PROGRAM-ID. name IS INITIAL` get a fresh WORKING-STORAGE at each call, and items in the LOCAL-STORAGE SECTION are always new. `CANCEL name` is accepted and does nothing in this release.

A program that cannot be found is an exception: the `ON EXCEPTION` statements are performed. If the CALL has no ON EXCEPTION phrase, a missing program is ignored and the statement after it is performed.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. CALLER.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 N        PIC 99 VALUE 0.
       01 TOTAL    PIC 999.
       PROCEDURE DIVISION.
           CALL "COUNTER" USING N.
           CALL "COUNTER" USING N.
           DISPLAY "after two calls N = " N.
           CALL "COUNTER" USING BY CONTENT N.
           DISPLAY "after BY CONTENT N = " N.
           CALL "ADDER" USING 30 12 RETURNING TOTAL.
           DISPLAY "30 + 12 = " TOTAL.
           CALL "FRESH".
           CALL "FRESH".
           CALL "NOSUCH" USING N
               ON EXCEPTION DISPLAY "NOSUCH is not available"
           END-CALL.
           STOP RUN.
       END PROGRAM CALLER.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. COUNTER.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 CALLS     PIC 99 VALUE 0.
       LINKAGE SECTION.
       01 OUT-N     PIC 99.
       PROCEDURE DIVISION USING OUT-N.
           ADD 1 TO CALLS.
```

```
           MOVE CALLS TO OUT-N.
           GOBACK.
       END PROGRAM COUNTER.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. ADDER.
       DATA DIVISION.
       LINKAGE SECTION.
       01 A         PIC 99.
       01 B         PIC 99.
       01 R         PIC 999.
       PROCEDURE DIVISION USING A B RETURNING R.
           COMPUTE R = A + B.
           EXIT PROGRAM.
       END PROGRAM ADDER.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. FRESH IS INITIAL.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 K         PIC 9 VALUE 0.
       PROCEDURE DIVISION.
           ADD 1 TO K.
           DISPLAY "FRESH: K = " K.
           GOBACK.
       END PROGRAM FRESH.

$ ./demo
after two calls N = 02
after BY CONTENT N = 02
30 + 12 = 042
FRESH: K = 1
FRESH: K = 1
NOSUCH is not available
```

Figure 70. CALL and the argument modes

The program names are looked up in upper case. A name is limited to 38 characters. All programs of a translation, including nested ones, are known by name to all the others.

User-defined functions

A function is a program with `FUNCTION-ID. name.` in place of PROGRAM-ID. It has a LINKAGE SECTION, a header `PROCEDURE DIVISION USING parameter ... RETURNING result`, and ends with `GOBACK` and `END FUNCTION name.`. A program that calls a function lists it in the REPOSITORY paragraph of the CONFIGURATION SECTION and uses it as `FUNCTION name (arguments)` in an expression, as in the example. The result item is not cleared when the function is entered; a function that builds a string with STRING first moves spaces to the result, as the example does.

```
       IDENTIFICATION DIVISION.
       FUNCTION-ID. DOUBLE-IT.
       DATA DIVISION.
       LINKAGE SECTION.
       01 N         PIC 9(4).
       01 R         PIC 9(5).
       PROCEDURE DIVISION USING N RETURNING R.
           COMPUTE R = N * 2.
           GOBACK.
       END FUNCTION DOUBLE-IT.

       IDENTIFICATION DIVISION.
```

```
       FUNCTION-ID. GREET.
       DATA DIVISION.
       LINKAGE SECTION.
       01 NM        PIC X(10).
       01 OUT-TXT   PIC X(20).
       PROCEDURE DIVISION USING NM RETURNING OUT-TXT.
           MOVE SPACES TO OUT-TXT.
           STRING "Hello, " FUNCTION TRIM (NM) "!"
               DELIMITED BY SIZE INTO OUT-TXT.
           GOBACK.
       END FUNCTION GREET.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. USEIT.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           FUNCTION DOUBLE-IT
           FUNCTION GREET.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 V         PIC 9(5).
       PROCEDURE DIVISION.
           COMPUTE V = FUNCTION DOUBLE-IT (21) + 1.
           DISPLAY V.
           DISPLAY FUNCTION GREET ("World").
           IF FUNCTION DOUBLE-IT (5) > 9 DISPLAY "greater" END-IF.
           STOP RUN.
       END PROGRAM USEIT.

$ ./demo
00043
Hello, World!
greater
```

Figure 71. User-defined functions

Separate modules

A program need not be linked into the main program. The option `-M` translates and compiles a source to a **module**, a shared library that registers its programs. A CALL to a program that is not in the executable searches for the module at run time: for the name `taxcalc` the directories of the environment variable `COB_LIBRARY_PATH`, a list separated by colons, and then the current directory are searched, for `taxcalc.so` and `libtaxcalc.so` in lower case and in the case of the name. The module is loaded once. It needs the dynamic loader, so it is not available on Windows, and it can be switched off by the environment variable `COB_NODL`.

```
      * main.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. SHOP.
      * The main program calls the subprogram TAXCALC.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 PRICE        PIC 9(5)V99 VALUE 250.00.
       01 TAX          PIC 9(5)V99.
       01 TAX-RATE     PIC 9V999   VALUE 0.077.
       01 SHOWN        PIC ZZ,ZZ9.99.
       PROCEDURE DIVISION.
           CALL "TAXCALC" USING PRICE TAX-RATE TAX.
           MOVE TAX TO SHOWN.
           DISPLAY "tax on 250.00 at 7.7 percent: " SHOWN.
           STOP RUN.

      * tax.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. TAXCALC.
      * Computes a tax amount, rounded to the cent.
       DATA DIVISION.
       LINKAGE SECTION.
       01 L-AMOUNT     PIC 9(5)V99.
       01 L-RATE       PIC 9V999.
       01 L-TAX        PIC 9(5)V99.
       PROCEDURE DIVISION USING L-AMOUNT L-RATE L-TAX.
           COMPUTE L-TAX ROUNDED = L-AMOUNT * L-RATE.
           GOBACK.

$ cobol -M -c tax.cob -o taxcalc.so
$ cobol -c main.cob -o prog
$ COB_LIBRARY_PATH=. ./prog
tax on 250.00 at 7.7 percent:     19.25
```

Figure 72. A main program and a separate module

Calling C functions

When the name of a CALL is not the name of a COBOL program, the run-time system looks for a C function of that name in the executable, first as written and then in lower case. The function can have up to eight arguments, each of the size of a machine word. A BY REFERENCE argument arrives as the address of the data (a `char *`, which is not terminated by a NUL byte), a BY VALUE argument as an integer. The value that the function returns goes to the RETURNING item and also to `RETURN-CODE`; since `RETURN-CODE` is the exit status of the program, the example ends with `MOVE ZERO TO RETURN-CODE`.

The C file is named on the command line of the translator after the COBOL source; the translator hands it to the C compiler.

```
/* cfn.c */
/* cfn.c -- C functions called from COBOL with CALL.
   BY REFERENCE arguments arrive as addresses of the COBOL data,
   BY VALUE arguments as ints. The data is not NUL terminated. */
#include <ctype.h>
#include <string.h>

int c_gcd(int a, int b)
{
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
```

```
    }
    return a;
}

/* converts the first n characters of s to upper case, in place;
   returns the number of letters changed */
int c_upcase(char *s, int n)
{
    int i, changed = 0;
    for (i = 0; i < n; i++)
        if (islower((unsigned char)s[i])) {
            s[i] = (char)toupper((unsigned char)s[i]);
            changed++;
        }
    return changed;
}

/* the length of s without trailing blanks */
int c_trimlen(char *s, int n)
{
    while (n > 0 && s[n - 1] == ' ')
        n--;
    return n;
}
```

```
      * main.cob
       IDENTIFICATION DIVISION.
       PROGRAM-ID. CALLC.
      * Calls the C functions in cfn.c.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 A            PIC S9(4) COMP-5 VALUE 84.
       01 B            PIC S9(4) COMP-5 VALUE 36.
       01 RESULT       PIC S9(9) COMP-5.
       01 TEXT-1       PIC X(20) VALUE "mixed Case text".
       01 LEN          PIC S9(9) COMP-5.
       PROCEDURE DIVISION.
           CALL "c_gcd" USING BY VALUE A B
               RETURNING RESULT.
           DISPLAY "gcd(84, 36) = " RESULT.
           MOVE 20 TO LEN.
           CALL "c_upcase" USING TEXT-1 BY VALUE LEN
               RETURNING RESULT.
           DISPLAY TEXT-1 " (" RESULT " letters changed)".
           CALL "c_trimlen" USING TEXT-1 BY VALUE LEN
               RETURNING RESULT.
           DISPLAY "length without trailing blanks: " RESULT.
           MOVE ZERO TO RETURN-CODE.
           STOP RUN.

$ cobol -c main.cob cfn.c -o prog
$ ./prog
gcd(84, 36) = +000000012
MIXED CASE TEXT      (+000000012 letters changed)
length without trailing blanks: +000000015
```

Figure 73. Calling C functions from COBOL

Chapter 13. Object-Oriented COBOL

cobol translates the object-oriented COBOL of COBOL 2002: classes with data and methods, inheritance and the INVOKE statement. A class is one source unit that begins with `CLASS-ID. name [INHERITS parent].` and ends with `END CLASS name.`. Inside the class, the `OBJECT.` part declares the data of each object (in its own WORKING-STORAGE SECTION) and the methods of the objects; the `FACTORY.` part declares data and methods that belong to the class itself. Each part ends with `END OBJECT.` or `END FACTORY.`

Classes and methods

A **method** is a small program inside the class. It begins with `METHOD-ID. name [OVERRIDE].`, may have a DATA DIVISION with a LINKAGE SECTION (parameters and result) and a LOCAL-STORAGE SECTION, has a `PROCEDURE DIVISION [USING ...] [RETURNING ...]`, and ends with `END METHOD name.` The data of the OBJECT part are visible in all its methods, and keep their values between calls: they are the **instance data**. Method names are compared in upper case.

The classes that a program uses must be named in the REPOSITORY paragraph: `CLASS name AS "external-name".` An item that holds a reference to an object is declared `USAGE OBJECT REFERENCE [class]`. Its value is null at the start.

Form	Meaning
`INVOKE class "NEW" RETURNING obj`	Creates an object of the class. The method NEW of every class creates the object and gives its data their VALUE clauses.
`INVOKE obj "METHOD" USING a ... RETURNING r`	Calls a method of the object. The method name is a literal or a data item.
`INVOKE class "METHOD" ...`	Calls a method of the FACTORY part of the class.
`INVOKE obj "FREE"`	Releases the object.
`INVOKE SELF "METHOD"`	The object that is running the method calls another of its own methods.
`INVOKE SUPER "METHOD"`	Calls the method of the parent class that the current method overrides.
`SET obj TO NULL`	The reference no longer points to an object.
`... ON EXCEPTION ... END-INVOKE`	Performed when the method is not found.

Figure 74. Object-oriented statements and words

A class that `INHERITS` another has all its methods and data, and can redefine a method with the same name by writing `OVERRIDE`. An object of the subclass can be used wherever an object of the parent class is expected. There is no multiple inheritance. The program may have up to 64 classes.

The first example has a class `ACCOUNT` with a balance and three methods, a subclass `SAVINGS` that adds interest with a method of its own and the data item `RATE`, and a program that uses both. The subclass calls the methods of its parent through `SELF`.

```
       IDENTIFICATION DIVISION.
       CLASS-ID. ACCOUNT.
      * A bank account class with a savings subclass.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS ACCOUNT AS "Account".
       OBJECT.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 BALANCE        PIC S9(7)V99 VALUE ZERO.
       PROCEDURE DIVISION.
       METHOD-ID. DEPOSIT.
       DATA DIVISION.
```

```
       LINKAGE SECTION.
       01 AMT            PIC 9(7)V99.
       PROCEDURE DIVISION USING AMT.
           ADD AMT TO BALANCE.
       END METHOD DEPOSIT.
       METHOD-ID. WITHDRAW.
       DATA DIVISION.
       LINKAGE SECTION.
       01 AMT            PIC 9(7)V99.
       01 OK-FLAG        PIC X.
       PROCEDURE DIVISION USING AMT RETURNING OK-FLAG.
           IF AMT > BALANCE
               MOVE "N" TO OK-FLAG
           ELSE
               SUBTRACT AMT FROM BALANCE
               MOVE "Y" TO OK-FLAG
           END-IF.
       END METHOD WITHDRAW.
       METHOD-ID. GET-BALANCE.
       DATA DIVISION.
       LINKAGE SECTION.
       01 OUT-BAL        PIC S9(7)V99.
       PROCEDURE DIVISION RETURNING OUT-BAL.
           MOVE BALANCE TO OUT-BAL.
       END METHOD GET-BALANCE.
       END OBJECT.
       END CLASS ACCOUNT.

       IDENTIFICATION DIVISION.
       CLASS-ID. SAVINGS INHERITS ACCOUNT.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS ACCOUNT AS "Account"
           CLASS SAVINGS AS "Savings".
       OBJECT.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 RATE           PIC 9V99 VALUE 0.05.
       PROCEDURE DIVISION.
       METHOD-ID. ADD-INTEREST.
       DATA DIVISION.
       LOCAL-STORAGE SECTION.
       01 CURRENT-BAL    PIC S9(7)V99.
       01 INTEREST       PIC 9(7)V99.
       PROCEDURE DIVISION.
           INVOKE SELF "GET-BALANCE" RETURNING CURRENT-BAL.
           COMPUTE INTEREST ROUNDED = CURRENT-BAL * RATE.
           INVOKE SELF "DEPOSIT" USING INTEREST.
       END METHOD ADD-INTEREST.
       END OBJECT.
       END CLASS SAVINGS.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. BANK.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS ACCOUNT AS "Account"
           CLASS SAVINGS AS "Savings".
       DATA DIVISION.
       WORKING-STORAGE SECTION.
```

```
       01 ACC            USAGE OBJECT REFERENCE ACCOUNT.
       01 SAV            USAGE OBJECT REFERENCE SAVINGS.
       01 AMOUNT         PIC 9(7)V99.
       01 OK-FLAG        PIC X.
       01 BAL            PIC S9(7)V99.
       01 SHOWN          PIC -Z,ZZZ,ZZ9.99.
       PROCEDURE DIVISION.
           INVOKE ACCOUNT "NEW" RETURNING ACC.
           MOVE 500.00 TO AMOUNT.
           INVOKE ACC "DEPOSIT" USING AMOUNT.
           MOVE 800.00 TO AMOUNT.
           INVOKE ACC "WITHDRAW" USING AMOUNT RETURNING OK-FLAG.
           DISPLAY "withdraw 800: " OK-FLAG.
           MOVE 120.50 TO AMOUNT.
           INVOKE ACC "WITHDRAW" USING AMOUNT RETURNING OK-FLAG.
           DISPLAY "withdraw 120.50: " OK-FLAG.
           INVOKE ACC "GET-BALANCE" RETURNING BAL.
           MOVE BAL TO SHOWN.
           DISPLAY "balance " SHOWN.
           INVOKE SAVINGS "NEW" RETURNING SAV.
           MOVE 1000.00 TO AMOUNT.
           INVOKE SAV "DEPOSIT" USING AMOUNT.
           INVOKE SAV "ADD-INTEREST".
           INVOKE SAV "GET-BALANCE" RETURNING BAL.
           MOVE BAL TO SHOWN.
           DISPLAY "savings " SHOWN.
           INVOKE ACC "FREE".
           INVOKE SAV "FREE".
           STOP RUN.

$ ./account
withdraw 800: N
withdraw 120.50: Y
balance        379.50
savings      1,050.00
```

Figure 75. Classes, objects and inheritance

Factory methods and overriding

A method in the FACTORY part belongs to the class and not to an object. The data of the FACTORY part exist once, so they can count the objects that were made. A factory method called `NEW` replaces the standard one: it calls `INVOKE SUPER "NEW" RETURNING r` to make the object, and can then prepare it. The next class numbers its objects with a factory counter, and a subclass overrides a method and calls the original through SUPER.

```
       IDENTIFICATION DIVISION.
       CLASS-ID. TICKET.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS TICKET AS "Ticket".
       FACTORY.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 ISSUED        PIC 99 VALUE 0.
       PROCEDURE DIVISION.
       METHOD-ID. NEW.
       DATA DIVISION.
       LINKAGE SECTION.
       01 R             USAGE OBJECT REFERENCE.
       PROCEDURE DIVISION RETURNING R.
```

```
           INVOKE SUPER "NEW" RETURNING R.
           ADD 1 TO ISSUED.
           INVOKE R "SET-NUMBER" USING ISSUED.
       END METHOD NEW.
       METHOD-ID. HOW-MANY.
       DATA DIVISION.
       LINKAGE SECTION.
       01 N             PIC 99.
       PROCEDURE DIVISION RETURNING N.
           MOVE ISSUED TO N.
       END METHOD HOW-MANY.
       END FACTORY.
       OBJECT.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 MY-NO          PIC 99 VALUE 0.
       PROCEDURE DIVISION.
       METHOD-ID. SET-NUMBER.
       DATA DIVISION.
       LINKAGE SECTION.
       01 N              PIC 99.
       PROCEDURE DIVISION USING N.
           MOVE N TO MY-NO.
       END METHOD SET-NUMBER.
       METHOD-ID. DESCRIBE.
       PROCEDURE DIVISION.
           DISPLAY "ticket " MY-NO.
       END METHOD DESCRIBE.
       END OBJECT.
       END CLASS TICKET.

       IDENTIFICATION DIVISION.
       CLASS-ID. VIP-TICKET INHERITS TICKET.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS TICKET AS "Ticket"
           CLASS VIP-TICKET AS "VipTicket".
       OBJECT.
       PROCEDURE DIVISION.
       METHOD-ID. DESCRIBE OVERRIDE.
       PROCEDURE DIVISION.
           DISPLAY "VIP" WITH NO ADVANCING.
           DISPLAY " " WITH NO ADVANCING.
           INVOKE SUPER "DESCRIBE".
       END METHOD DESCRIBE.
       END OBJECT.
       END CLASS VIP-TICKET.

       IDENTIFICATION DIVISION.
       PROGRAM-ID. TICKETS.
       ENVIRONMENT DIVISION.
       CONFIGURATION SECTION.
       REPOSITORY.
           CLASS TICKET AS "Ticket"
           CLASS VIP-TICKET AS "VipTicket".
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 T1            USAGE OBJECT REFERENCE TICKET.
       01 T2            USAGE OBJECT REFERENCE TICKET.
       01 HOW-MANY-T     PIC 99.
       PROCEDURE DIVISION.
```

```
           INVOKE TICKET "NEW" RETURNING T1.
           INVOKE VIP-TICKET "NEW" RETURNING T2.
           INVOKE T1 "DESCRIBE".
           INVOKE T2 "DESCRIBE".
           INVOKE TICKET "HOW-MANY" RETURNING HOW-MANY-T.
           DISPLAY HOW-MANY-T " tickets issued".
           INVOKE T1 "UNKNOWN-METHOD"
               ON EXCEPTION DISPLAY "no such method"
           END-INVOKE.
           SET T1 TO NULL.
           INVOKE T1 "DESCRIBE".
           STOP RUN.
       END PROGRAM TICKETS.

$ ./demo
ticket 01
VIP ticket 02
02 tickets issued
no such method
cobol: runtime error: INVOKE on a null object reference
```

Figure 76. Factory methods and SUPER

The last INVOKE uses a null reference. It stops the program with the run-time error `INVOKE on a null object reference` and exit status 1. A class that is not found by its external name gives the error `class not found`.

Chapter 14. Diagnostics and Run-Time Errors

Messages of the translator

The translator stops at the first error. It writes one line in the form `cobol: file: line n: message`, where file is the name of the source file (the copybook, for an error in a copybook) and n the line, and ends with the return code 8. The C compiler then is not started. The next table lists the messages that you will meet most often; a word of the source that is meant is added at the end of the message.

Message	Cause
`unknown data name X`	A name is used that no data description declares. Often the cause is a name that was cut at column 72 (Chapter 3).
`undefined paragraph or section X`	PERFORM or GO TO names a paragraph that does not exist.
`duplicate paragraph X`, `duplicate section X`	A paragraph or section name is used twice in one program.
`item needs a PICTURE: X`	An elementary item has no PIC and no usage that does without one.
`unknown data description clause near X`	A clause is not known, or the entry is not ended by a period.
`copybook not found: X`	The COPY file is not in the directory of the source, in the current directory or in a directory of the option `-I`.
`unsupported statement X`	A statement of the standard that is not implemented: ALTER, ENTRY, JSON, XML, GENERATE, INITIATE, TERMINATE, VALIDATE and the report writer statements.
`statement expected near X`	A word that cannot begin a statement stands where a statement is expected; often a missing operand or a period in the wrong place.
`unexpected word in statement near X`	A statement has a word that does not belong to it.
`integer expected near X`	A number is needed, as in OCCURS, where a data name or a constant is not accepted.
`FD without SELECT: X`	A file is described in the FILE SECTION but not named in FILE-CONTROL.
`unknown name in file entry X`	A SELECT names a data item that is not declared, as in ASSIGN TO or RECORD KEY.
a message of the C compiler	The C compiler reports an error in the generated file; the return code is 8. It is rare, and it is a defect of the translator; see Chapter 15.

Figure 77. Messages of the translator

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 TOTAL-AMOUNT PIC 9(4).
       PROCEDURE DIVISION.
           MOVE 1 TO TOTAL-AMOUNT.
           ADD 1 TO TOTAL-AMOUNTS.
           STOP RUN.

$ cobol -c demo.cob -o demo
cobol: demo.cob: line 8: unknown data name TOTAL-AMOUNTS
```

Figure 78. An undeclared name

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       PROCEDURE DIVISION.
       MAIN.
           PERFORM CLEAN-UP.
           STOP RUN.
       CLEANUP.
           DISPLAY "done".

$ cobol -c demo.cob -o demo
cobol: demo.cob: line 5: undefined paragraph or section CLEAN-UP
```

Figure 79. An undefined paragraph

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 DOC PIC X(40).
       PROCEDURE DIVISION.
           JSON PARSE DOC INTO DOC.
           STOP RUN.

$ cobol -c demo.cob -o demo
cobol: demo.cob: line 7: unsupported statement JSON
```

Figure 80. A statement that is not implemented

The line number counts all lines of the file, including comment lines and blank lines, as the file was written.

Run-time errors

The generated program stops on an error that it finds while it runs. It writes `cobol: runtime error: message` to the standard error file, and the exit status is 1. COBOL programs usually check for such errors by themselves with ON SIZE ERROR, FILE STATUS and the other phrases of the statements; the message appears only if the program leaves an error unhandled.

Message	Cause
`file name (status = nn)`	A file statement failed with a status of 10 or more, and the file has no FILE STATUS item, no declarative and no AT END or INVALID KEY phrase (Chapter 10).
`subscript out of range`	A subscript is zero or larger than the table (only when `COB_CHECK` is set).
`reference modification out of range`	The start or the length of a reference modification is outside the item (only when `COB_CHECK` is set).
`INVOKE on a null object reference`	INVOKE of an object reference that has no object.
`class not found`, `method called without an object`, `too many classes`	Errors of the object system (Chapter 13).
`user-defined function not found`, `intrinsic function not implemented`	A function call that cannot be done (Chapters 9 and 12).
`PERFORM nested too deeply`	More than 128 PERFORMs are active at the same time.
`GO TO without a target`	A GO TO with no paragraph name was executed.
`INSPECT: too many phrases`	An INSPECT with more than eight phrases.
`too many programs`, `out of memory`	The tables of the run-time system are full, or the memory is exhausted.

Figure 81. Run-time errors

The tests of subscripts and of reference modification are off by default, because they slow the program down. They are switched on for one run by the environment variable COB_CHECK. With it, a subscript that is zero or larger than the number of elements, stops the program at once:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 TBL.
          05 EL      PIC 9 OCCURS 3 TIMES.
       01 I          PIC 9 VALUE 5.
       PROCEDURE DIVISION.
           MOVE 7 TO EL (I).
           DISPLAY "stored without a check".
           STOP RUN.

$ COB_CHECK=1 ./demo; echo "exit status $?"
cobol: runtime error: subscript out of range
exit status 1
```

Figure 82. Checking subscripts

The option -check of the translator is accepted, but in this release it does not change the generated program; use COB_CHECK.

The exit status

The exit status of a program is the value of RETURN-CODE at STOP RUN, 0 if the program did not change it. A run-time error gives 1. The translator gives 4 for a wrong command line and 8 for an error in the source or in the C compilation. In a shell script the status can be tested with $?; the examples in Chapter 6 show MOVE 3 TO RETURN-CODE.

Chapter 15. Implementation Notes and Limits

The decimal engine

All arithmetic on numeric data is done by a decimal engine that is part of the run-time header. A number is a sign, a string of at most 80 decimal digits and a scale, the number of digits after the decimal point. Addition, subtraction and multiplication are exact. A division is carried to 34 decimal places and then cut, or rounded if ROUNDED was asked for. The engine takes its operands from the fields in whatever usage they have (zoned, packed, binary, edited) and stores the result back with the alignment, truncation or rounding that the target needs.

The consequences for the programmer are these. An intermediate result with more than 80 digits is a size error. Because a division carries 34 places, the dividend of a division should have no more than 46 digits before the decimal point; for larger dividends the result is not correct (see the list of defects below). Only COMP-1 and COMP-2 fields, the floating-point functions and exponentiation with a fractional exponent are done in the binary `double` type, with the usual 15 to 16 correct digits.

Data representation

Item	Convention
Zoned decimal (DISPLAY)	One character per digit. The sign of an unsigned-separate field is combined with the last digit: the character code of the digit is raised by 0x40 for a negative value on an ASCII machine (zone D on EBCDIC), and a positive value is the plain digit (zone C on EBCDIC for a signed field).
Packed decimal (COMP-3)	Two digits per byte, the sign in the last half byte (C or F positive, D negative).
Binary (COMP, COMP-4, BINARY)	Big-endian, 2, 4, 8 or 16 bytes according to the number of digits. The value is truncated to the digits of the PICTURE. With the translator option `-binsize 1-2-4-8` a field of one or two digits has one byte.
Binary (COMP-5, COMP-X)	As above in the byte order of the machine.
BINARY-CHAR to BINARY-DOUBLE	The native type; the value wraps around at the size of the type.
COMP-1, COMP-2	C `float` and `double`. DISPLAY writes them with up to 15 digits.
Alphanumeric	One byte per character in the character set of the machine; the sort and comparison order is the order of that set.
NATIONAL	Two bytes per character.
Pointers and object references	Eight bytes.
Initial values	Numbers are zero, alphanumeric items are blanks, pointers are null, unless VALUE says otherwise.

Figure 83. Representation of data

Limits

Item	Limit
Digits of a numeric field in arithmetic	80
Decimal places of a quotient	34
Length of a program name	38 characters
Nested PERFORMs active at once	128
OCCURS levels in the initialization of a table	3
Keys of a SORT	16
Alternate keys of an indexed file	8

Item	Limit
Nesting of COPY	20
Phrases of one INSPECT	8
Arguments of a call of a C function	8
Classes of a program	64

Figure 84. Limits of the translator and the run-time system

What is not implemented

The translator reports `unsupported statement` for these statements: ALTER, ENTRY, JSON, XML, VALIDATE, GENERATE, INITIATE, TERMINATE, COMMIT, ROLLBACK, RAISE, RESUME, SUPPRESS, UNLOCK. The REPORT SECTION, SCREEN SECTION and COMMUNICATION SECTION are skipped without being translated, so a program that needs them does not work. The following are accepted in the source and ignored: the paragraphs SOURCE-COMPUTER, OBJECT-COMPUTER and I-O-CONTROL, the ALPHABET clause and other special names that change the collating sequence, LOCK MODE, RESERVE, SHARING, LINAGE (the item LINAGE-COUNTER is not maintained), the clause EXTERNAL, the word COMMON, and the statement CANCEL. All programs of a translation can be called by all others, whether nested or not. The clauses TYPEDEF, TYPE and PROPERTY are not implemented.

Known defects

The following points are defects of this release. Each is described with the way to avoid it.

Defect	What to do
A sentence that begins with `EXIT SECTION` is taken for a section header.	Put another statement of the same sentence before it, or end the section by performing it.
The option `-check` and the variable `COB_CHECK` enable run-time checks only for subscripts and reference modification.	Test other conditions in the program.
`SEARCH ALL` scans the table from its first element.	It is correct for a table in key order but not faster than `SEARCH`.
`ALTER` is not implemented, and a bare `GO TO.` stops the program when it is reached.	Use `GO TO ... DEPENDING ON` or a switch item.
The generated C lines are wrapped at 71 columns; a very long string literal is split in pieces.	This is meant for CMS, see Appendix C.

Figure 85. Known defects

Two further points are limits of the design. `SEARCH ALL` scans the table from its first element; it is correct for a table in key order but not faster than `SEARCH`. An indexed file is held in memory between OPEN and CLOSE, so it should be of moderate size and always be closed.

Files of the translator

The repository holds the translator in `src`, as fifteen files that `tools/amalg.sh` joins into the single file `cobol_all.c`, and the parts of the run-time header, `rt0_head.h` to `rt8_obj.h`, which `tools/mkrt.sh` joins into `cobrt.h`. The tests are in `tests` and the examples of this manual in `examples`. The Makefile builds the translator with `-std=c89 -pedantic` and runs the tests.

Appendix A. Words to Avoid

COBOL has many reserved words. cobol does not keep a full list of them: the translator knows the words of the statements, the words that appear inside statements, and the words of data descriptions, and it needs them to find where a statement ends. A user-defined name that is one of these words is a source of errors, because the word is taken as part of the language and not as a name. The error message is usually `unexpected word in statement`, as in the example. Every word that begins with `END-` is also reserved. Besides, avoid the names of the figurative constants (ZERO, SPACE, HIGH-VALUE, LOW-VALUE, QUOTE, ALL), of the special registers (RETURN-CODE, TALLY, ADDRESS, LENGTH, NULL, SELF, SUPER) and of the intrinsic functions.

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. DEMO.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 COUNT PIC 9.
       PROCEDURE DIVISION.
           MOVE 1 TO COUNT.
           STOP RUN.

$ cobol -c demo.cob -o demo
cobol: demo.cob: line 7: unexpected word in statement near 'COUNT'
```

Figure 86. A reserved word used as a name

The tables are made from the lists in the translator: the first are the words that begin a statement, the second the words inside statements, the third the words of data descriptions. A name that is made by adding a prefix, such as `WS-COUNT` or `COUNT-1`, is always safe.

Word	Word	Word	Word
ACCEPT	EVALUATE	MULTIPLY	START
ADD	EXIT	NEXT	STOP
ALLOCATE	FREE	OPEN	STRING
ALTER	GENERATE	PERFORM	SUBTRACT
CALL	GO	RAISE	SUPPRESS
CANCEL	GOBACK	READ	TERMINATE
CLOSE	IF	RELEASE	UNLOCK
COMMIT	INITIALIZE	RESUME	UNSTRING
COMPUTE	INITIATE	RETURN	USE
CONTINUE	INSPECT	REWRITE	VALIDATE
DELETE	INVOKE	ROLLBACK	WRITE
DISPLAY	JSON	SEARCH	XML
DIVIDE	MERGE	SET	
ENTRY	MOVE	SORT	

Figure 87. Statement words

Word	Word	Word	Word
ADDRESS	END	NO	RUN
ADVANCING	EQUAL	NOT	SECTION
AFTER	ERROR	OF	SENTENCE
ALL	EXCEPTION	OMITTED	SEQUENCE

Word	Word	Word	Word
ALSO	EXTEND	ON	SEQUENTIAL
AND	FALSE	ONLY	SHARING
AS	FILE	OPTIONAL	SIZE
ASCENDING	FIRST	OR	STATUS
AT	FOR	OTHER	TALLY
BACKWARD	FORWARD	OUTPUT	TALLYING
BEFORE	FROM	OVERFLOW	TEST
BY	GIVING	PAGE	THAN
CHARACTERS	GREATER	POINTER	THEN
COLLATING	I-O	PREVIOUS	THROUGH
CONTENT	IN	PROCEDURE	THRU
CONVERTING	INITIAL	PROGRAM	TIMES
CORR	INPUT	RANDOM	TO
CORRESPONDING	INTO	RECORD	TRUE
COUNT	INVALID	RECORDS	UNTIL
DELIMITED	IS	REFERENCE	UPON
DELIMITER	KEY	RELATIVE	USING
DEPENDING	LEADING	REMAINDER	VALUE
DESCENDING	LESS	REPLACING	VARYING
DIVISION	LINE	RETURNING	WHEN
DUPLICATES	LINES	REVERSED	WITH
DYNAMIC	LOCK	REWIND	
ELSE	NEXT	ROUNDED	

Figure 88. Words inside statements

Word	Word	Word	Word
ALIGNED	COMP-X	FUNCTION-POINTER	PROCEDURE-POINTER
BASED	COMPUTATIONAL	GLOBAL	PROPERTY
BINARY	COMPUTATIONAL-1	INDEX	REDEFINES
BINARY-CHAR	COMPUTATIONAL-2	IS	RENAMES
BINARY-DOUBLE	COMPUTATIONAL-3	JUST	SIGN
BINARY-LONG	COMPUTATIONAL-4	JUSTIFIED	SYNC
BINARY-SHORT	COMPUTATIONAL-5	LEADING	SYNCHRONIZED
BLANK	CONSTANT	NATIONAL	TRAILING
COMP	DISPLAY	OBJECT	TYPE
COMP-1	DYNAMIC	OCCURS	TYPEDEF
COMP-2	EXTERNAL	PACKED-DECIMAL	USAGE
COMP-3	FLOAT-EXTENDED	PIC	VALUE
COMP-4	FLOAT-LONG	PICTURE	VALUES
COMP-5	FLOAT-SHORT	POINTER	

Figure 89. Words of data descriptions

Appendix B. GnuCOBOL Compatibility

GnuCOBOL is a free COBOL compiler that also makes C. cobol was written independently of it, but follows its conventions in many places, so that most programs for one work with the other. This appendix lists where the two agree and where they differ, as far as the author has been able to check.

Topic	cobol
Command line	The options `-c`, `-x`, `-o`, `-free`, `-fixed`, `-I`, `-D` and `-M` have the meaning they have for `cobc`; `-x` is accepted as a synonym of `-c`. The other options of `cobc` are not provided. The compiler is found by `-cc`, by `COBOL_CC`, or by searching for gcc, cc, clang and tcc.
Source format	Fixed and free format, the directives `>>SOURCE FORMAT`, `>>DEFINE`, `>>IF`, `>>ELIF`, `>>ELSE` and `>>END-IF`, and the literals X"", H"", Z"", B"", BX"", N"" and NX"" are accepted. `*>` comments work in free format only.
Data	The usages COMP-1 to COMP-5, COMP-X, BINARY, PACKED-DECIMAL, the BINARY-xxx and FLOAT-xxx types, national items, pointers and object references. Binary fields are big-endian and cut to the picture, as with the default of GnuCOBOL (`binary-truncate`).
Display of numbers	DISPLAY of a numeric item writes the sign and an explicit decimal point where the picture has a V. A program whose output is compared with that of another compiler should write numbers through edited items.
Arithmetic	Exact decimal arithmetic with 80 digits (GnuCOBOL: 38 digits per field). The functions that use floating point are computed in `double`.
Intrinsic functions	About one hundred functions; the date, string, statistics and national functions of GnuCOBOL are there. FUNCTION-ID functions and `FUNCTION ALL INTRINSIC` work.
Files	Sequential, line sequential, relative and indexed files with alternate keys, FILE STATUS and declaratives. The indexed file is a flat file in key order, not a BDB or VBISAM file, so a data file of one system cannot be read by the other. Names are mapped by `DD_name`, `dd_name` and `name` in the same order as GnuCOBOL.
CALL	Static and dynamic calls, `COB_LIBRARY_PATH`, `-M` modules and calls of C functions with BY VALUE arguments, but not the `cob_` library functions of GnuCOBOL (`cob_init`, `cob_field`). The library routines `CBL_*` and `C$*` are not provided; `CALL "SYSTEM"` is there.
Debugging	There is no `-debug` run-time checking in the sense of GnuCOBOL; the option adds the lines with D in column 7. Subscript checks are enabled by `COB_CHECK`.
Not available	The report writer, the screen section, `ALTER`, JSON and XML statements, the `TYPEDEF` clause and the `LINAGE` counter.

Figure 90. Comparison with GnuCOBOL

A program that uses only the features of Chapters 3 to 13 translates with `cobol` and compiles with `cobc -x`. The reverse is true if the program stays within the same limits. For a portable program, keep to fixed format, avoid extensions such as `COMP-X` with other sizes, and keep file names in literals.

Appendix C. Running on VM/370 CMS

cobol was designed to run on the VM/370plus system with GCC, as described in the Preface. This appendix gives the steps for CMS. They follow the conventions of the GCC for VM/370 and have **not** been tried: the translator and the generated programs have been built and tested only with a Linux C compiler in C89 mode. Where the behaviour depends on the C library (the mapping of file names, the arguments of the program), read the notes of the library you choose.

1. Send `cobol_all.c` to CMS as `COBOL C A`, and `cobrt.h` as `COBRT H A`. The lines of the translator and of `cobrt.h` are no longer than 71 columns, so records of 80 characters hold them without loss.
2. Compile with `GCC31 COBOL C A (CMS` to use GCCLIB, or without `CMS` to use PDPCLIB. The result is `COBOL TEXT A`.
3. Make the library available with `GLOBAL TXTLIB GCCLIB` (or `PDPCLIB`), then `LOAD COBOL` and `START`.
4. Run the translator on a COBOL source file. It writes the C file next to it and, unless you give `-c`, stops there; compile that file in the same way as in step 2. The generated file includes `cobrt.h`, so `COBRT H` must be on an accessed disk.

Lines of the generated C program. The 71-column rule holds for the translator, but not for the C program it writes: some generated lines are longer than 80 columns (lines of about 155 characters occur in the tests). The comment in the source that speaks of wrapping the output for CMS describes a feature that is not there. When the C file is sent to CMS or created there, use a record length of at least 160 (variable records, `LRECL 160`), and not fixed 80-column records, or the long lines will be cut and the C compiler will report errors.

Limits on CMS. The C library of CMS is not expected to have `dlopen`, so the dynamic modules of Chapter 12 and the calls of C functions by name do not work; link the called programs into one load module instead, as `cobol -c main.cob sub.cob` does on Linux. Only `main` is an external name in the generated program; all other functions and variables are `static`, so the limit of eight characters for external names does not matter. Names of COBOL files are given to the C library as they are written in the program, so a program that is to run on both systems should take its file names from the environment (`DD_name`) or from a data item.

Take care of the character set. The run-time system uses the character set of the C compiler; on CMS this is EBCDIC, and the sign zones of numbers, the collating sequence and the results of the intrinsic functions `ORD` and `CHAR` are those of EBCDIC.